MoA All the Way: From ψ-Indexed Semantics to Verified Compiler ASTs and Ω Machine Mappings
A Comprehensive SC27 Working Manuscript

Lenore M. Mullin Gaëtan Hains
Working research draft; author list and affiliations to confirm

November 3, 2026 — verified compiler integration revision 41 (integrated Step 108 working draft)

Abstract

Mathematics of Arrays (MoA) provides a theory-first method for deriving computations over shaped arrays and mapping them to machine architectures. Prior research derived and implemented array and transformer attention computations by hand and evaluated implementations using NSF ACCESS computing resources. The present research aims to mechanize those mathematical transformations. We describe a compiler path from shaped array expressions to a semantic abstract syntax tree (AST), ψ reduction, architecture-independent Denotational Normal Form (DNF), sequential Operational Normal Form (ONF) using explicit γ addressing, and dimension-lifted ONF based on machine shape. Exact index coverage and source-level resource predictions are distinguished from empirical timing models and the longer-term objective of machine-derived performance prediction. A preliminary UAlbany DGX attention study calibrated an empirical, MoA-work-informed model on three sequence lengths and froze its predictions before two holdout lengths. Among four candidate mappings, the selected mapping was the fastest measured in both holdouts; its absolute relative timing errors were 0.22% and 4.83%. These results demonstrate a working mechanization and validation protocol, not yet a proof of exact execution-time prediction from machine shape. The original AST rewrite chain is retained. The portable realization agenda uses OpenMP for shared-memory execution, OpenACC for accelerator offload, and MPI for distributed communication. Historical device-specific generated code is retained as evidence of an earlier restricted prototype, not presented as the portability mechanism. Finite tests cover deterministic rooted communication, port-constrained scheduling, and conditional homogeneous and heterogeneous link-time predictions. The latest November 2026 compiler integration reports 234 passing regression tests, including independent topology, scheduling, heterogeneous communication-time, directed-link contention, and candidate-selection audits. In a six-coordinate example, the audited selection lowers conditional predicted communication time from 12.0648 to 12.0576 microseconds without changing logical traffic or stage count. These tests do not constitute end-to-end distributed execution, measured network speedup, or a global optimality proof.

1 Research motivation and historical path

MoA begins with the mathematical meaning of a shaped array computation, not an architecture-specific implementation. Its central methodology is to formulate the array expression, reduce its ψ indexing semantics, obtain an architecture-independent DNF, map that normal form into a sequential ONF, and only then dimension-lift the operational form onto a shaped machine. The DNF is invariant under a change of target machine; mapping and execution structure may change while the computed array remains the same.

The research program predates the compiler: attention and other array computations were formulated, reduced, programmed, and experimentally tested by hand. The hand-derived experimental program used ACCESS resources including Stampede, Expanse, Bridges, Purdue Anvil, and NCSA Delta. The Delta

AMD GPU experiments extended the study to another accelerator architecture. A separate Chameleon Cloud proposal award supported Intel GPU energy experiments. These resource histories are distinct from the present University at Albany DGX compiler validation. These experiments informed increasingly explicit machine-shape descriptions, now developed to ten fields in the predictive compiler work. The goal of the current collaboration is to mechanize the hand-verified sequence and test whether the compiler reproduces the semantic, resource, and eventually performance behavior of the reference implementations. This paper distinguishes established hand-derived results from newer compiler-generated evidence.

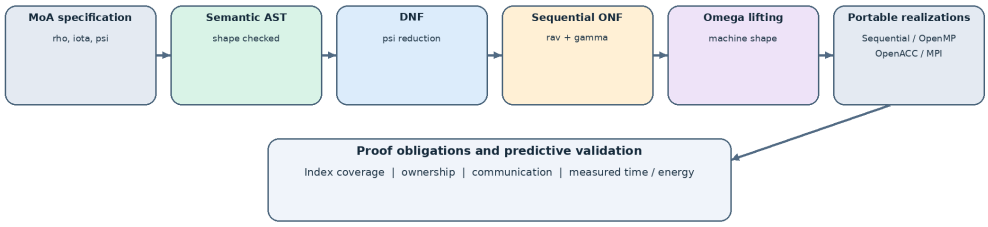


Figure 1: Proposed mechanized MoA transformation and validation path. ψ reduction produces architecture-independent DNF; γ addressing yields sequential ONF; Ω lifting changes execution shape. The machine descriptor enters the mapping and prediction stages.

2 Foundations: shape, index domain, and ψ

For an array A, ρA is its shape, $\iota(\rho A)$ generates its valid index domain, and $\vec{\imath}\,\psi\, A$ denotes ψ selection. The element count and scalar dimensionality are

$$\tau(A) = \prod(\rho A), \qquad \delta(A) = \tau(\rho A). \tag{1}$$

Index completeness is expressed as $(\iota(\rho A))\ \psi\ A \equiv A$. These are properties of MoA arrays and not machine addresses. Vectors are rank-one arrays; matrices are rank-two arrays. Architecture-specific storage and traversal are introduced later.

3 MoA all the way: the constructive proof architecture

The proposed compiler accepts a *semantically defined array computation*, regardless of whether its source syntax originated in Fortran, MATLAB, Python, Julia, C, or another front end. A front end is responsible for preserving the defined meaning of the supported source-language subset; the common MoA machinery begins at that semantic boundary. Neither a language name nor a target instruction set appears in the mathematical DNF. Consequently, any supported source languages whose programs have the same defined array semantics obtain the same MoA DNF and the same mathematically derivable optimization opportunities; only their front-end representation and machine-specific realization differ.

Let E be the MoA expression and $\mathcal{R}_\psi$ its index-composition rewrite relation. Write $E \to_\psi^* D$ for a finite sequence of ψ reductions ending in DNF D. The Church–Rosser property means that diverging reductions can be joined; where the relevant reductions terminate in a unique canonical normal form, equivalent expressions have the same DNF. For MoA programs expressed over finite, shape-bounded index domains, the shape components supply the finite upper bounds on index composition and traversals. Thus the shape-bounded MoA reductions and computations considered here terminate; this statement concerns the defined MoA domain and does not assert termination of arbitrary unrestricted source-language control flow. We use Church–Rosser and the resulting common DNF as the mathematical

equivalence criterion.

$$E_1 \to_\psi^* D \quad \wedge \quad E_2 \to_\psi^* D \quad \Longrightarrow \quad E_1 \equiv_{\mathrm{MoA}} E_2. \tag{2}$$

A machine description $\mathcal{M}$ supplies shaped resources and associated component values (capacities, hierarchy, transfer capabilities, costs, and constraints). For a feasible mapping μ the construction is

$$E \xrightarrow{\psi^*} D \xrightarrow{\text{1–1 indexed mapping}} \mathrm{ONF}_\gamma(D) \xrightarrow{\Omega_\mu} \mathrm{ONF}^\Omega_{\gamma,\mathcal{M}} \xrightarrow{\text{faithful instruction realization}} I_{\mathcal{M}}. \tag{3}$$

The proof is compositional: index composition establishes DNF; a layout function γ represents the same indexed elements at addresses; Ω lifts and partitions those elements without omission or duplication; machine instructions implement the resulting ONF. Each arrow carries its own mathematical invariant. Thus *specification proves realization* for supported mappings and instruction semantics by construction, rather than by comparing arbitrary optimized binaries after the fact. Floating-point reassociation, reduction order, exceptional values, and communication must be governed by an explicit numerical semantics or a stated error relation.

4 A visible attention AST rewrite sequence

Consider one attention head with $\rho Q = \langle n, d\rangle$, $\rho K = \langle m, d\rangle$, $\rho V = \langle m, d_v\rangle$. The last-two-axis transpose is mandatory: $\rho(K^{T_{\mathrm{last2}}}) = \langle d, m\rangle$. In particular,

$$\langle j, k\rangle\psi K^{T_{\mathrm{last2}}} = \langle k, j\rangle\psi K, \qquad 0 \le j < d,\ 0 \le k < m. \tag{4}$$

This is a *semantic* transpose rule, not an instruction to allocate a transposed copy. The small AST beside each mathematical stage is intended to make the corresponding hand derivation visually checkable. Figure 2 displays the original typed expression tree.

Apply $\langle i, v\rangle\psi$ to the output, then recursively compose indices through matrix multiplication, transpose, scaling, and row normalization. For the score contraction the critical rewrite is

$$\langle i, k\rangle\psi(Q \,\mathrm{MM}\, K^{T_{\mathrm{last2}}}) \to_\psi \sum_{j=0}^{d-1} (\langle i, j\rangle\psi Q)(\langle j, k\rangle\psi K^{T_{\mathrm{last2}}}) \tag{5}$$

$$\to_\psi \sum_{j=0}^{d-1} (\langle i, j\rangle\psi Q)(\langle k, j\rangle\psi K). \tag{6}$$

The second arrow is index composition through transpose. It explains why, for fixed i, k and row-major Q, K, both j-indexed operands are traversed contiguously. A different logical layout can be realized by a regular strided transfer when the machine supports it. The adjacent compact AST displays the intermediate ψ-indexed tree.

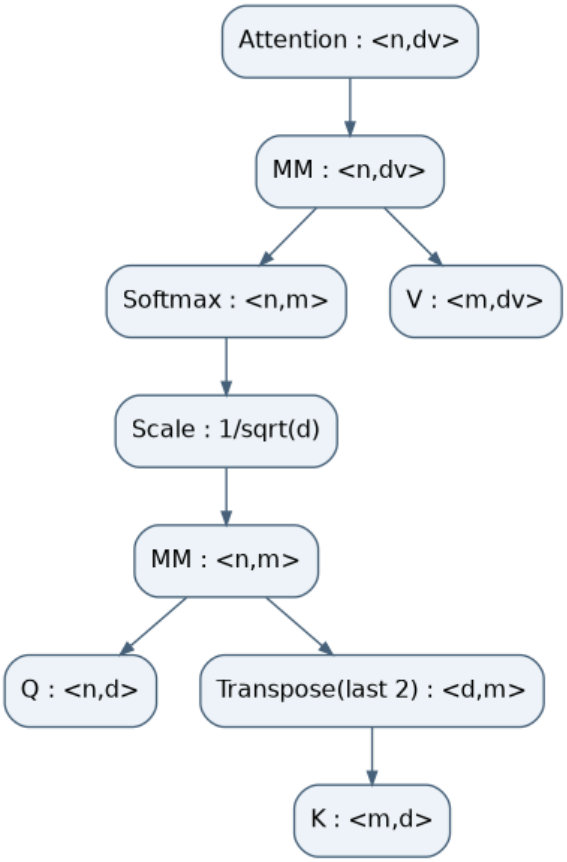
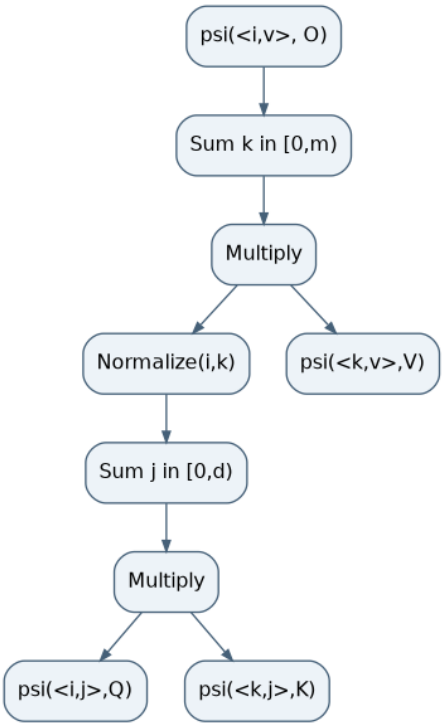


Figure 2: Beginning AST: attention as shape-checked array operators. The K transpose changes its logical index order; its storage need not change.

Indexed contraction: $s(i,k) = d^{-1/2} \sum_j Q[i,j]K[k,j]$.



4.1 Cartesian-indexed DNF: the end of semantic reduction

The result is not merely a loop nest: it is a function on a Cartesian index domain. With $\mathcal{I} = \{(i,k,j,v) \mid 0 \le i < n,\ 0 \le k < m,\ 0 \le j < d,\ 0 \le v < d_v\}$, define

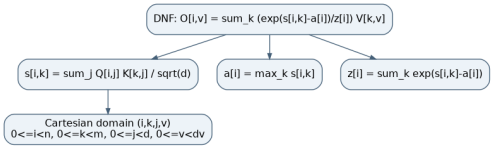
$$s(i,k) = \frac{1}{\sqrt{d}} \sum_{j=0}^{d-1} (\langle i,j\rangle \psi Q)(\langle k,j\rangle \psi K), \tag{7}$$

$$a(i) = \max_{0 \le k < m} s(i,k), \qquad z(i) = \sum_{k=0}^{m-1} e^{s(i,k)-a(i)}, \tag{8}$$

$$\langle i,v\rangle \psi O = \sum_{k=0}^{m-1} \frac{e^{s(i,k)-a(i)}}{z(i)} (\langle k,v\rangle \psi V). \tag{9}$$

All domains, shapes, and selections are explicit. This is the architecture-independent reference to which the subsequent operational forms must correspond. The adjacent compact AST displays the end of ψ reduction.

DNF: each output $\langle i,v\rangle \psi O$ is determined by the displayed Cartesian-indexed sums, maxima, and selections.



4.2 DNF to sequential ONF: addressing is the realization

For a layout family γ_λ (row-major, column-major, or another supported layout), addressing is a function of an index and the *shape of the indexed array*:

$$\vec{\imath}\psi X = (\mathrm{rav}\, X)[\gamma_\lambda(\vec{\imath}; \rho X)]. \tag{10}$$

For row-major arrays, $\gamma_{\text{row}}(\langle i,j\rangle;\langle n,d\rangle) = id + j$ and $\gamma_{\text{row}}(\langle k,j\rangle;\langle m,d\rangle) = kd + j$. In contrast, $\gamma_{\text{col}}(\langle i,j\rangle;\langle n,d\rangle) = i + nj$. In particular $\langle k,j\rangle\psi K = (\text{rav}\, K)[\gamma_{\text{row}}(\langle k,j\rangle;\rho K)]$. For fixed k, its j sequence has $(\text{start},\text{stop},\text{stride}) = (kd,(k+1)d,1)$ in elements (exclusive stop). The corresponding Q sequence is $(id,(i+1)d,1)$. Other valid mappings can derive a regular nonunit stride and use supported DMA, vector, or hierarchical transfers; contiguity is a useful cost characteristic, not an axiom of universal optimality.

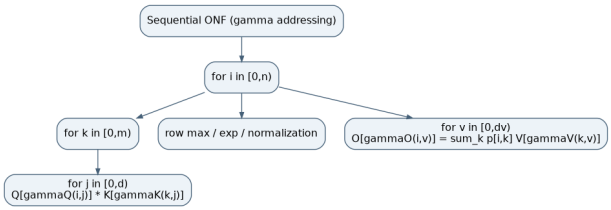


Figure 3: Sequential ONF AST: the DNF's exact index selections become γ-addressed operations. Loop ordering is a mapping choice; the Cartesian-indexed DNF remains unchanged.

4.3 Sequential to parallel and distributed ONF by dimension lifting

Let $L = \lceil n/P \rceil$ and $I_p = \{pL + u \mid 0 \le u < L,\ pL + u < n\}$. For $n > 0$ and $P > 0$,

$$\{0,\ldots,n-1\} = \biguplus_{p=0}^{P-1} I_p. \tag{11}$$

The quotient–remainder decomposition proves unique ownership of each output query i. A scheduling function $\sigma(p)$ assigns logical partitions to workers; P need not equal the number of threads, GPUs, or nodes. The same lifted ONF can therefore be realized as an OpenMP loop, a GPU grid, an MPI partition, or a heterogeneous assignment without altering DNF.

If K, V are replicated, each query owner can compute its output independently. If keys and values are sharded, the score maximum, exponential denominator, and weighted numerator require communication. For disjoint key shards H_r, define local stable summaries $(a_r, z_r, u_{r,v})$ where $a_r = \max_{k\in H_r} s(i,k)$, $z_r = \sum_{k\in H_r} e^{s(i,k)-a_r}$, and $u_{r,v} = \sum_{k\in H_r} e^{s(i,k)-a_r}(\langle k,v\rangle\psi V)$. Combining two summaries gives

$$a = \max(a_1, a_2), \tag{12}$$

$$z = e^{a_1-a} z_1 + e^{a_2-a} z_2, \tag{13}$$

$$u_v = e^{a_1-a} u_{1,v} + e^{a_2-a} u_{2,v}, \qquad O[i,v] = u_v/z. \tag{14}$$

This composition reproduces the global stable-normalization formula in exact arithmetic and permits a tree reduction across distributed resources. Floating-point reproducibility and exceptional cases require explicit conventions. The mapping must also establish the placement and transfer of Q, K, V across memory levels, and ensure each output is written exactly once.

4.4 Constructive equivalence and performance selection

The proof obligation is a commuting correspondence: every ψ selection in DNF is realized by exactly the appropriate γ-addressed value, every output index belongs to one lifted owner, and any distributed reduction implements the same mathematical reduction. The generated instructions must faithfully implement the lifted ONF. These are proof obligations of the construction, not experimental claims. Among mappings satisfying them and the resource constraints, the compiler can compare predicted time and energy, potentially identifying a Pareto frontier rather than a single simultaneously optimal mapping. Measured runtime and energy remain necessary to validate physical predictions.

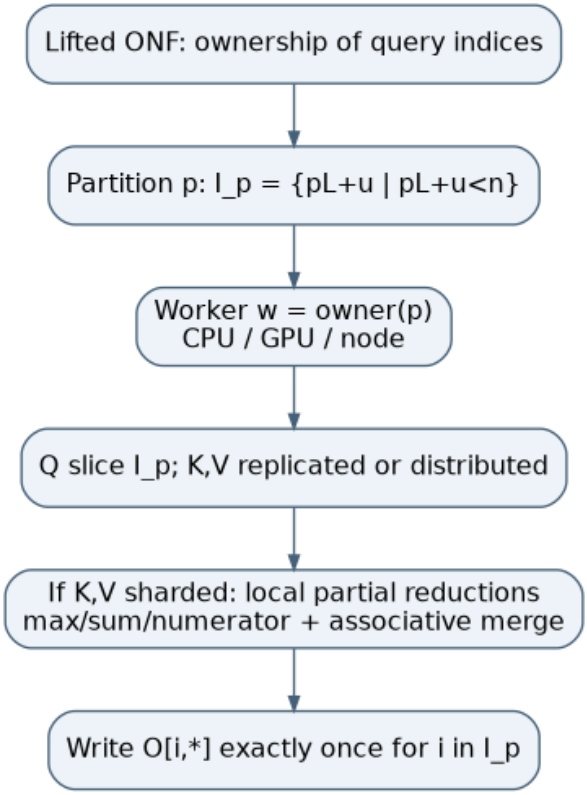


Figure 4: Lifted ONF AST: the Cartesian query domain becomes disjoint ownership domains; the machine-shape mapping chooses CPU, GPU, and distributed workers. Replication or sharding of K, V determines whether distributed summary reductions are needed.

5 Worked attention expression and AST

Let $\rho Q = \langle B, N, D \rangle$, $\rho K = \langle B, N, D \rangle$, and $\rho V = \langle B, N, D_v \rangle$. A representative attention expression has indexed scores, stable probabilities, and output

$$S_{b,i,j} = D^{-1/2} \sum_{d=0}^{D-1} (\langle b, i, d \rangle \, \psi \, Q)(\langle b, j, d \rangle \, \psi \, K), \tag{15}$$

$$m_{b,i} = \max_{0 \le j < N} S_{b,i,j}, \tag{16}$$

$$P_{b,i,j} = \frac{\exp(S_{b,i,j} - m_{b,i})}{\sum_{k=0}^{N-1} \exp(S_{b,i,k} - m_{b,i})}, \tag{17}$$

$$\langle b, i, v \rangle \, \psi \, O = \sum_{j=0}^{N-1} P_{b,i,j} (\langle b, j, v \rangle \, \psi \, V). \tag{18}$$

This is an explanatory indexed expansion of the attention semantics. Section 6 develops the explicit component equations and a row-fused ONF using the available SC27 derivation source. The current baseline generator still materializes probabilities and a score workspace.

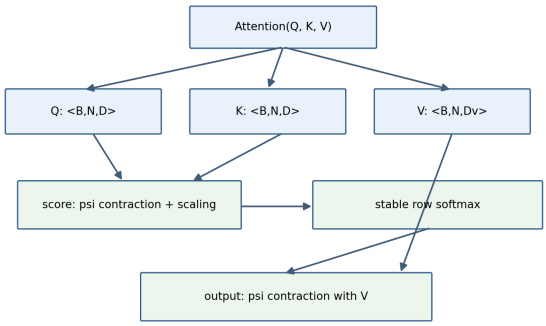


Figure 5: Schematic attention semantic AST: shaped inputs, ψ-based contraction, stable normalization, and output contraction. The production compiler diagram should enumerate actual AST node types and rewrite rules.

6 Paper 1 mathematical path: input expression to reduced DNF

This section makes the attention derivation explicit rather than treating the computation as a black-box library call. The equations are grounded in the worked attention derivation in the earlier MoA attention derivation and SC27 working source used in preparing this draft. They are *not* presented as a verbatim transcription of the arXiv source; final notation and code alignment should be checked against the authoritative Paper 1 submission before publication.

6.1 Input array expression and typed semantic AST

For single-head unmasked attention, set $\rho Q = \langle n, d_k \rangle$, $\rho K = \langle m, d_k \rangle$, $\rho V = \langle m, d_v \rangle$, and $\rho O = \langle n, d_v \rangle$. The input expression is

$$O = \mathrm{softmax}(QK^{\mathsf{T}}/\sqrt{d_k})V. \tag{19}$$

The transpose in this surface expression is a semantic access transformation, not a requirement to materialize a transposed array. An AST must carry shapes and index extents at each node:

Attention(Softmax(Scale(Contract(Q, Transpose(K)))), V), with well-formedness constraints $\rho Q[1] = \rho K[1]$ and $\rho K[0] = \rho V[0]$. Figure 6 displays the rewrite stages.

6.2 ψ-reduced component semantics

Applying ψ to the score expression makes the contraction and its valid domain explicit:

$$s_{ik} = d_k^{-1/2} \sum_{j=0}^{d_k-1} (\langle i, j\rangle \; \psi \, Q)(\langle k, j\rangle \; \psi \, K), \quad 0 \le i < n, \; 0 \le k < m. \tag{20}$$

Stable normalization is given by

$$a_i = \max_{0 \le k < m} s_{ik}, \qquad z_i = \sum_{k=0}^{m-1} \exp(s_{ik} - a_i), \tag{21}$$

and the output by

$$\langle i, v\rangle \; \psi \, O = \sum_{k=0}^{m-1} \frac{\exp(s_{ik} - a_i)}{z_i} (\langle k, v\rangle \; \psi \, V), \qquad 0 \le v < d_v. \tag{22}$$

Equations (20)–(22) specify the architecture-independent component computation. They expose precisely which Q, K, and V elements are required and permit rectangular $n \neq m$ inputs. The maximum subtraction is algebraically neutral over exact real arithmetic but floating-point evaluation requires a stated tolerance and reduction-order policy.

6.3 A fused, memory-reduced operational derivation

A score matrix of shape $\langle n, m\rangle$ contains nm elements; separately materialized probabilities require another nm elements. For one query i, however, Eqs. (20)–(22) depend only on the m scores for that query. Therefore a sequential ONF can calculate $s_{i0}, \ldots, s_{i,m-1}$ in an m-element scratch array, compute a_i, z_i, and accumulate the d_v output components without storing the full $n \times m$ score or probability arrays. In fp64 the score scratch is $8m$ bytes per concurrently processed query, apart from runtime overhead and output storage. This is the reference *row-fused* memory bound; it must not be confused with a proof of the globally minimal memory bound for every attention variant.

An additional algebraic option, which must be checked against the specific Paper 1 code before being attributed to that implementation, is an online stable normalization. Define $m_r = \max_{0 \le k < r} s_{ik}$, $\ell_r = \sum_{k<r} \exp(s_{ik} - m_r)$, and $u_{r,v} = \sum_{k<r} \exp(s_{ik} - m_r)(\langle k, v\rangle \; \psi \, V)$. When score s_{ir} arrives, with $m' = \max(m_r, s_{ir})$, update

$$\ell_{r+1} = e^{m_r - m'} \ell_r + e^{s_{ir} - m'}, \tag{23}$$

$$u_{r+1,v} = e^{m_r - m'} u_{r,v} + e^{s_{ir} - m'} (\langle r, v\rangle \; \psi \, V). \tag{24}$$

The invariant follows by factoring $e^{-m'}$ from the sum over the first $r+1$ scores. After processing all m keys, $\langle i, v\rangle \; \psi \, O = u_{m,v}/\ell_m$. This alternative avoids an m-score scratch array but has distinct numerical and implementation characteristics; it is shown as a derivable transformation, not as an assertion about the current compiler's generated kernel.

6.4 DNF to γ-addressed sequential ONF

For row-major storage, the shape-dependent addressing instances are

$$\gamma_{\mathrm{row}}(\langle i, j\rangle; \rho Q) = id_k + j, \qquad \gamma_{\mathrm{row}}(\langle k, j\rangle; \rho K) = kd_k + j, \tag{25}$$

$$\gamma_{\mathrm{row}}(\langle k, v\rangle; \rho V) = kd_v + v, \qquad \gamma_{\mathrm{row}}(\langle i, v\rangle; \rho O) = id_v + v. \tag{26}$$

Then $(\langle i, j\rangle \, \psi \, Q)$ is realized as $(\mathrm{rav}\, Q)[\gamma_{\mathrm{row}}(\langle i, j\rangle; \rho Q)]$, and similarly for the other arrays. This changes the realization, not the DNF's index semantics. The sequential ONF loops over i, then k, j to form scores, reduces the row maximum and denominator, and loops over v, k to produce each output. The row-fused realization retains only one query's score scratch at a time.

6.5 Lifted ONF, exact coverage, and resource allocation

Partition query indices into P nonoverlapping domains I_p satisfying $\biguplus_{p=0}^{P-1} I_p = \{0, \ldots, n-1\}$. For the concrete contiguous partition rule $L = \lceil n/P \rceil$, let $I_p = \{pL + u \mid 0 \le u < L,\ pL + u < n\}$. Every valid query index occurs in exactly one domain because Euclidean quotient and remainder with divisor L are unique. This establishes query ownership; preservation of score reductions, scratch ownership, and output writes must also be verified. The number of partitions P is independent of the number of OpenMP threads T. A resource model must account for at least T concurrent scratch arrays of length m for a thread-local row-fused implementation, and for communication when K, V are distributed.

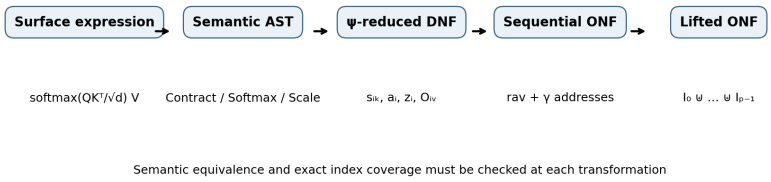


Figure 6: Detailed attention rewrite chain: the surface expression becomes a shaped semantic AST; ψ reduction exposes exact component dependencies; the DNF is realized with γ addresses and then lifted to disjoint query partitions. The diagram is a conceptual transformation trace, not an export of the compiler's internal AST.

6.6 Compiler acceptance test against the hand derivation

For every supported input shape, a compiler-generated ONF must match the independently implemented ψ-indexed DNF and hand-derived ONF within a declared floating-point criterion. The compiler must also report the exact covered query domain, per-partition work, temporary allocation, and machine mapping. Only after those checks should predicted time, power, and energy be evaluated on held-out hardware configurations. In particular, the v13 DGX predictor reported later in this paper measures a materializing baseline; it is *not* a performance result for the row-fused or online-memory-reduced realization above.

7 ψ reduction, DNF, and sequential ONF

ψ reduction makes the index domain and element dependencies explicit. DNF fixes the computation independent of storage layout and processor allocation. For a row-major operational realization, γ can

map a three-dimensional array index to an address offset:

$$\gamma_{\text{row}}(\langle b, i, d\rangle; \rho Q) = ((bN + i)D + d), \quad \langle b, i, d\rangle \, \psi \, Q = (\text{rav}\, Q)[\gamma_{\text{row}}(\langle b, i, d\rangle; \rho Q)]. \tag{27}$$

For example, the ONF for an output element uses $(\text{rav}\, O)[\gamma_{\text{row}}(\langle b, i, v\rangle; \rho O)] \leftarrow \sum_j (\text{rav}\, P)[\gamma_{\text{row}}(\langle b, i, j\rangle; \rho P)](\text{rav}\, V)[\gamma_{\text{row}}(\langle b, j, v\rangle; \rho V)]$, subject to the full preceding definitions of P. This address translation does not alter the DNF's meaning. The generated sequential code is then the baseline against which lifted implementations are checked.

8 Dimension lifting and shaped execution

For a query domain $I = \{0, \ldots, N-1\}$, a valid lifted partition must satisfy

$$I = \biguplus_{p=0}^{P-1} I_p, \qquad I_p \cap I_q = \varnothing \quad (p \neq q). \tag{28}$$

A concrete quotient/remainder construction may use $L = \lceil N/P \rceil$ and $i = pL + u$, with $0 \leq p < P$, $0 \leq u < L$, and the guard $i < N$. This proves coverage and disjointness of valid query indices. The partition count P is not the number of OpenMP threads T: multiple partitions can be scheduled by one thread.

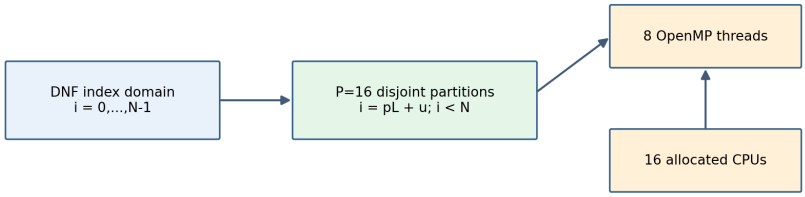


Figure 7: Query-domain lifting in the tested attention implementation. Sixteen logical partitions were executed with eight OpenMP threads inside a sixteen-logical-CPU Slurm allocation. This does not establish that sixteen-thread execution is slower; that configuration has not yet been tested.

9 Machine shape and prediction obligations

The deterministic machine-shape descriptor is under development as a ten-field characterization: compute units U, warp width W, shared memory M, measured reference time t, measured memory ceiling $n_{\max}$, total memory capacity M_{cap}, realization power P, execution-unit reachability X, compiler-directive configuration C, and reservation overhead R. Some fields are measured or realization-specific, so a deterministic machine shape does not automatically imply an analytically exact runtime prediction. The provisional CPU-only `MachineShape10` in the repository is a distinct implementation interface and must not be conflated with this research descriptor.

The current baseline attention kernel has source-level persistent arrays Q, K, V, P, O and a full score workspace. For 64-bit elements the corresponding source-level allocation model is

$$M_{\text{alloc}} = 8B(2ND + 2ND_v + 2N^2), \tag{29}$$

excluding allocator, runtime, and other transient overhead. The arithmetic multiplication count for the baseline score and output contractions is $W = BN^2(D + D_v)$. The current empirical predictor fits

$\widehat{T} = \alpha + \beta W_{\text{critical}}$ for each mapping, with W_{critical} estimated from static partition work. This is not yet a closed-form machine-shape-derived time model.

10 UAlbany DGX held-out attention validation

The experiment ran on `dgx03`, an AMD EPYC 7742 compute node, with a Slurm allocation of 16 logical CPUs and GCC 11.4.0 using `-O2 -fopenmp`. The measured affinity list was `60-63,124-127,188-191,252-255`. Double-precision baseline attention used $B = 1$, $D = D_v = 32$. The calibration lengths were $N \in \{96, 160, 256\}$, and the unseen holdouts were $N \in \{320, 448\}$. Each case used nine timed samples after four warmups. All reported P and O correctness errors were zero at the printed precision. The prediction certificate was frozen before validation with SHA-256

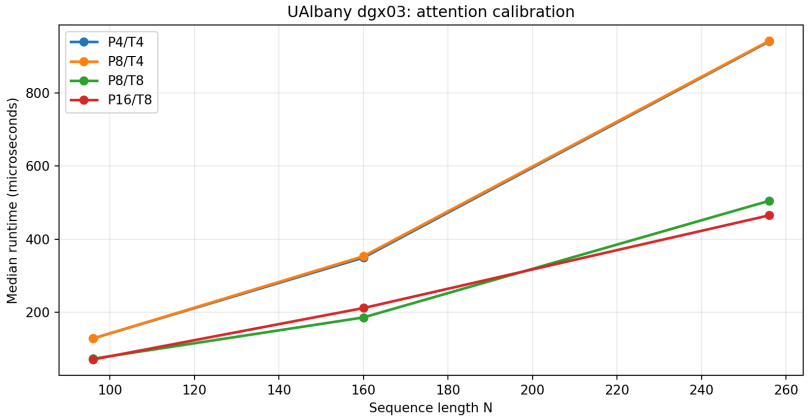


Figure 8: Measured calibration medians for the four partition/thread candidates. The legend distinguishes partitions P from threads T.

Table 1: Frozen-prediction validation, selected mapping $P = 16$, $T = 8$.

N	Predicted (μs)	Measured (μs)	Abs. error	Selection regret
320	723.635	722.060	0.22%	0%
448	1400.348	1471.371	4.83%	0%

`a73981eb98d7f8858b9676168a86369a334092693005e33ef8b4b196b5923e2d`. The measured candidate times at $N = 320$ were 1482.522, 1429.852, 727.610, and 722.060 μs; at $N = 448$ they were 2929.547, 2919.638, 1477.623, and 1471.371 μs, respectively, in candidate order $(4, 4), (8, 4), (8, 8), (16, 8)$. The $N = 448$ difference between the two eight-thread mappings is small; repeated runs and placement controls are needed to assess selection stability.

11 Mechanization milestones and limitations

The compiler must (i) preserve the shaped semantics of the input AST, (ii) produce a verifiable ψ-reduced DNF, (iii) map DNF one-to-one to sequential ONF with explicit γ, (iv) produce lifted ONFs whose index sets form an exact disjoint cover, (v) predict resource requirements from the shaped computation and target, and (vi) compare frozen performance predictions against measurements. The present attention timing model is empirical; its thread-load assignment approximates OpenMP static scheduling and does not independently identify memory, synchronization, or affinity costs. The present generated

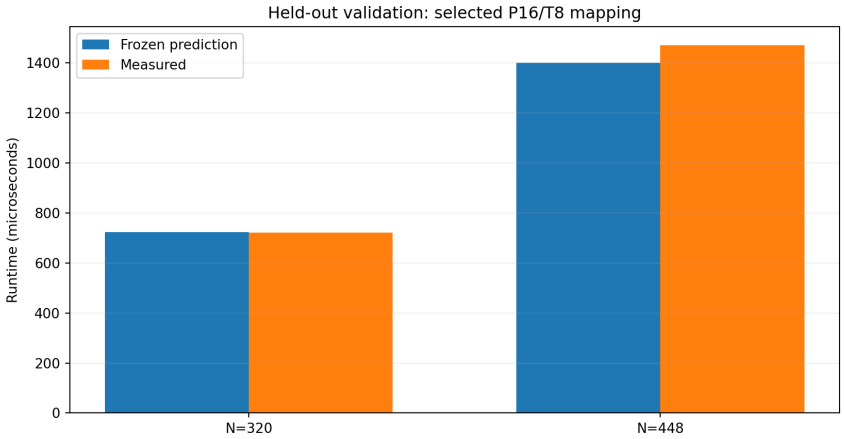


Figure 9: Frozen versus observed times for the selected mapping. Both selections were fastest among the four measured candidates, not necessarily globally optimal.

attention kernel is not the full memory-optimal Paper 1 implementation. A separate experiment must compare eight and sixteen OpenMP threads before asserting that the remaining eight allocated CPUs are redundant; subsequent experiments may test two concurrent eight-thread workloads and aggregate throughput.

12 MoA as a unified mathematics of computation, machines, and prediction

The long-term objective is not simply to compile one attention kernel for one processor. It is to derive, within MoA, semantically equivalent executable implementations for diverse configurations of coordinated heterogeneous resources and select a feasible mapping that optimizes execution time and energy according to an explicit objective. The computation and the available machine resources are both represented as shaped structures. MoA transformations are required to preserve the meaning of the computation while refining the operational placement, communication, and traversal of its elements.

Let E denote the input MoA expression, $D(E)$ its ψ-reduced DNF, and $\mathcal{M}$ a description of the available resource ensemble. For each feasible configuration $c \in \mathcal{C}(\mathcal{M})$, let $G(D(E), c)$ be generated executable code. The primary semantic obligation is

$$\mathrm{Sem}(G(D(E), c)) \equiv \mathrm{Sem}(E), \qquad c \in \mathcal{C}(\mathcal{M}), \tag{30}$$

where equivalence for floating-point programs must specify the arithmetic model, reduction order, and acceptable numerical tolerance. The compiler's search must also enforce memory capacity, index-domain coverage, data-dependency, reachability, and communication constraints. Thus semantic equivalence is not a consequence of measured speed; it is a separate proof obligation.

Time and energy are competing objectives. A precise statement of the design goal is to identify feasible nondominated configurations

$$\mathcal{P} = \left\{ c \in \mathcal{C}(\mathcal{M}) : \nexists c' \in \mathcal{C}(\mathcal{M}) \text{ with } T(c') \le T(c),\ E(c') \le E(c) \text{ and one strict} \right\}, \tag{31}$$

or minimize time subject to an energy budget (or vice versa). Here T and E are machine-dependent quantities that must be predicted and subsequently measured; neither is established solely by algebraic operation counts. The eventual compiler should generate independently checkable semantic and resource certificates and report uncertainty or error for physical predictions.

13 MoA all the way: four-paper compiler reproduction contract

The governing acceptance criterion is stronger than generating a numerically plausible attention kernel. For each of the four independently hand-derived research programs, the compiler must mechanically reproduce a traceable MoA derivation, an executable realization, and an independently testable prediction. The hand-derived programs and their published measurements are *reference evidence*, not compiler-generated results. No row/column-vector reinterpretation or architecture-specific intermediate mathematics is introduced: arrays retain their MoA shapes, and machine resources are shaped arrays too.

For each reference program p, let E_p be its shaped MoA specification, D_p the verified ψ-reduced DNF, O_p the γ-addressed ONF, and $L_p(\mathcal{M})$ the Ω-lifted ONF on shaped machine $\mathcal{M}$. The compiler's reproduction obligation is the certified chain

$$E_p \overset{\psi}{\Longrightarrow} D_p \xrightarrow{\gamma,\mathrm{rav},[\,]} O_p \xrightarrow{\Omega,\rho\mathcal{M}} L_p(\mathcal{M}) \longrightarrow G_p(\mathcal{M}),$$

with index-domain coverage, dependency preservation, legal ownership or reduction, resource feasibility, and a specified floating-point relation checked at each relevant transition. Before executing G_p, a machine-shape-based prediction and its assumptions must be frozen; measured results are then compared against that prediction and against the hand-derived reference. Numerical equivalence alone does not certify equivalent storage, communication, or predictive behavior.

Table 2: Four-paper reproduction contract. Entries describe required evidence, not completed compiler capabilities.

Hand-derived reference	MoA derivation and machine obligations	Reproduction evidence required
Forward attention (Paper 1)	Score contraction, stable normalization, value contraction; DNF/ONF, scratch elimination, Ω ownership	Generated full forward kernel, numerical and address checks, allocation/traffic certificate, frozen predictions
Backward attention (Paper 2)	Gradient dependencies and reductions; legal accumulation, addressing, and partitioning	Generated backward kernel, gradient reference comparison, reduction/communication certificate, frozen predictions
Decode attention (Paper 3)	KV-cache indexing, GQA/MQA shape relationships, cache ownership, OpenACC realization	Generated decode kernel, cache/index checks, transfer/communication accounting, frozen predictions
Transformer block (Paper 4)	RMSNorm, attention, gated feed-forward composition and fusion; intermediate lifetimes	Generated complete block, stagewise/end-to-end checks, storage and communication certificates, frozen predictions

A pass is recorded only when the compiler artifacts and independent verification logs are archived for the same specification, shape, numerical contract, and machine configuration as the hand-derived reference. Existing component regression tests are evidence for their *individual* obligations; they do not automatically promote any of the four rows to end-to-end reproduced status. The first integration target is complete forward attention; subsequent targets extend the same MoA chain rather than introducing new foundations for backward, decode, or block execution.

14 Experimental lineage and resource provenance

The research sequence is theory first: formulate attention in MoA; perform ψ reduction; derive the DNF and ONF by hand; dimension-lift onto a target machine; implement; measure; and use discrepancies to

refine the machine description. The current compiler effort seeks to mechanize this established manual sequence. The earlier implementations should not be described as compiler-generated.

Table 3: Research resources and their distinct roles. Individual configurations, award identifiers, and measured outcomes require cross-reference to the corresponding experiment logs and publications.

Resource	Research role
Stampede, Expanse, Bridges	ACCESS-supported heterogeneous performance research and hand-derived implementation studies.
Purdue Anvil	ACCESS execution and validation of hand-derived MoA algorithms.
NCSA Delta (including AMD GPU)	Accelerator experiments, including AMD GPU execution.
Chameleon Cloud (separate award)	Intel GPU energy experiments, separate from the ACCESS resource allocation.
University at Albany DGX	Present compiler-generated CPU attention calibration and frozen holdout validation.

15 Coordinated heterogeneous resource allocation

A heterogeneous resource ensemble can include CPU cores, GPU execution units, local and remote memories, network links, and communication/synchronization mechanisms. A machine-shape representation must encode the capacities and access relations relevant to a particular mapping. The compiler should derive partitioned ONFs that cover the complete DNF index domain, establish ownership or legal replication of intermediate data, and satisfy all inter-partition dependencies. For any proposed coordinated configuration, the resource model must account for computation, data movement, transfers, synchronization, placement, power, and energy.

The present experiment compared four configurations with four or eight OpenMP threads; it did *not* compare eight and sixteen threads. The observation that eight threads gave the best result among the tested candidates motivates, but does not prove, that unused allocated CPUs can profitably perform independent attention work. The sixteen logical partitions are not sixteen executing threads: eight OpenMP threads schedule the sixteen partitions. The other eight allocated logical CPUs may support a second independent attention invocation, a concurrent attention head or batch, or communication/data staging, provided the hardware topology and memory bandwidth permit it. A next experiment will compare one eight-thread workload, one sixteen-thread workload, and two concurrent eight-thread workloads under controlled affinity and NUMA placement, measuring latency, aggregate attention throughput, and energy where instrumentation is available. These are proposed experiments, not established gains.

16 A traceable AST-to-instruction construction

The reader must be able to inspect *each* AST rewrite, not only the initial and final states. Figures 10 and 11 therefore trace one *score-contraction component* of attention. This is deliberately a bounded subexpression of the full stable-attention derivation; the softmax and value contraction require their own dependency-aware lowering. Every edge names the mathematical transformation that licenses the next AST. These figures are proposed compiler intermediate representations, not machine-generated AST dumps.

AST 0 to AST 3: semantic reduction. Let $\rho Q = \rho K = \langle B, N, D \rangle$ and $\rho S = \langle B, N, N \rangle$. The score formulation is the MoA contraction of Q with the last-two-dimension transpose of K. The successive

nodes show selection, transpose-index composition, and the resulting Cartesian-indexed DNF:

$$\begin{aligned}\langle b,i,k\rangle\psi S &= D^{-1/2}\sum_{d=0}^{D-1}(\langle b,i,d\rangle\psi Q)(\langle b,d,k\rangle\psi K^{T_{\text{last 2}}}) \\ &= D^{-1/2}\sum_{d=0}^{D-1}(\langle b,i,d\rangle\psi Q)(\langle b,k,d\rangle\psi K).\end{aligned} \tag{32}$$

The optimization story begins with a precise choice: the surface expression contains a last-two-dimension transpose, but the compiler need not allocate a transposed copy. AST 0 records the shaped contraction; AST 1 exposes the selected component; AST 2 composes the transpose indices; AST 3 is the Cartesian-indexed DNF. Only after that mathematical reduction does AST 4 introduce layout-dependent addresses and AST 5 assign the same output components to machine resources.

Why the rewrite improves addressing. In row-major layout, the reduction over d visits $Q[b,i,d]$ and $K[b,k,d]$ with element stride one. A mistaken untransposed selection $K[b,d,k]$ would traverse the second-to-last dimension with stride D (where the index is valid) and would generally denote a different computation. The correct logical transpose and subsequent ψ reduction therefore expose contiguous source accesses without materializing a transpose. For alternative layouts, $\gamma(\vec{i};\rho X)$ exposes regular strides that can be mapped to suitable hardware transfer mechanisms.

The valid Cartesian output index domain is $[B]\times[N]\times[N]$, and the reduction domain is $[D]$. In row-major storage the d traversal of both operands is contiguous; other γ families may give regular strides. No materialized transpose is required.



Figure 10: Compact AST progression for one attention score component. Transpose-index composition converts the logical transpose into direct $K[b,k,d]$ selection; the resulting contiguous reduction is then addressed with the shape-dependent γ family and assigned to machine-shape components. This is an illustrative AST trace, not compiler-generated output.

AST 4 to AST 6: constructive machine instantiation. The sequential ONF replaces every selection with $\text{rav}(X)[\gamma(\vec{i};\rho X)]$. Dimension lifting assigns the Cartesian output domain to machine-shape components. For disjoint output ownership, define $f_c : [B]\times[N]\times[N] \to \iota(\rho\mathcal{M}_c)$, with $\mathcal{D}_m = f_c^{-1}(m)$. The proof obligation is $\bigcup_m \mathcal{D}_m = [B]\times[N]\times[N]$ with pairwise disjoint $\mathcal{D}_m$; if the reduction domain itself is partitioned, an additional explicit combination operation is required. The same DNF admits sequential, OpenMP, OpenACC, and MPI/Open MPI realizations, each lowering the same indexed computation according to the machine shape and its legal ownership and communication constraints.

Instruction-level correspondence. For a CPU-owned output component, the reduction body is instantiated as `load Q[gamma(<b,i,d>;rho Q)]`, `load K[gamma(<b,k,d>;rho K)]`, `multiply/add or`

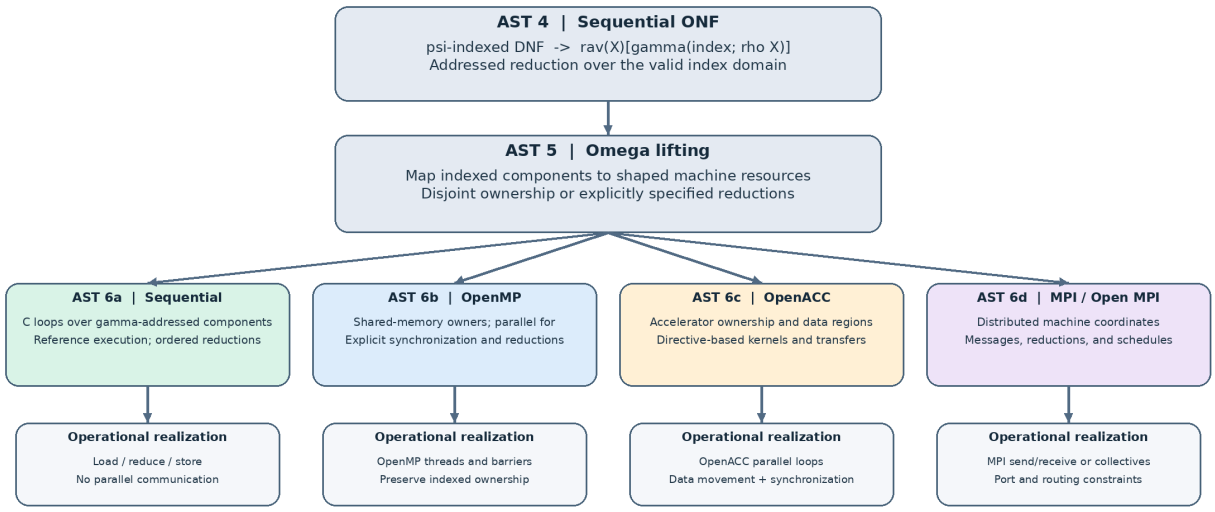


Figure 11: AST 4 → AST 5 → four realization branches: sequential, OpenMP, OpenACC, and MPI (with Open MPI as an implementation). All branches share the same Cartesian DNF and addressed ONF, with distinct ownership, transfer, reduction, and synchronization obligations. The diagram illustrates the portable target architecture, not four independently verified compiler emitters.

`fma`, followed by `store S[gamma(<b,i,k>;rho S)]`. A GPU-owned component uses corresponding device loads, arithmetic, and stores inside a kernel, with its block/thread index decoded into the lifted Cartesian index. A coordinated CPU–GPU realization additionally instantiates necessary copies or DMA transfers, stream or event dependencies, and a completion barrier. A device-specific backend must prove that these actual instructions implement the lifted ONF and respect its numerical contract; an FMA is not automatically bitwise equivalent to a separate multiply and add.

End-to-end attention. The same stepwise trace must subsequently be repeated for the maximum reduction, exponentiation, normalization, and the P–V contraction, or for the corresponding fused online-softmax recurrence from Paper 1. These operations cannot be replaced by the score-only AST without losing dependencies. The figures establish the *format of a checkable trace* and the explicit score-contraction example, not a claim that the full memory-optimal Paper 1 kernel has already been mechanically generated.

17 AST-to-instruction evidence and the portable realization objective

This section places code immediately after the AST sequence in Figures 10 and 11. It instantiates the *score contraction only*; the complete stable-softmax and value contraction of Paper 1 remain distinct derivations. Let $\rho Q = \rho K = \langle B, N, D\rangle$ and $\rho S = \langle B, N, N\rangle$. The shared Cartesian DNF is

$$\langle b, i, j\rangle \psi S = D^{-1/2} \sum_{d=0}^{D-1} (\langle b, i, d\rangle \psi Q)(\langle b, j, d\rangle \psi K). \tag{33}$$

AST 4 instantiates the row-major member of the γ family: $\gamma_{\text{row}}(\langle a, b, c\rangle; \langle n_0, n_1, n_2\rangle) = (an_1 + b)n_2 + c$. The general definition remains $\vec{i}\psi X = (\text{rav}\, X)[\gamma(\vec{i}; \rho X)]$; the code below is a specific row-major realization, not a restriction on MoA.

17.1 AST 4: shape-dependent address instantiation

Source listing not bundled in the preserved v38 archive: `attention_cpu.c`, lines 9–12. This historical excerpt is intentionally not reconstructed or fabricated.

Here $\gamma(\langle b, i, d\rangle; \rho Q)$ and $\gamma(\langle b, j, d\rangle; \rho K)$ generate the two operand addresses. The logical last-two-dimension transpose of K is eliminated by index composition; no physical transpose is allocated.

17.2 AST 5a–6a: CPU ownership and C/OpenMP instructions

Source listing not bundled in the preserved v38 archive: `attention_cpu.c`, lines 15–32. This historical excerpt is intentionally not reconstructed or fabricated.

Each OpenMP iteration owns one element of the Cartesian output domain. The reduction d is local to its owner, and all outputs are disjoint. The generated instruction classes are address calculation, load, multiply/add, scale, and store.

17.3 AST 5b–6b: portable accelerator ownership with OpenACC

The same lifted ONF assigns each output component to one owner. The following is a *schematic OpenACC realization* of the score contraction, not evidence of a separately tested compiler emitter:

```
#pragma acc parallel loop collapse(3) present(Q[0:B*N*D],K[0:B*N*D],S[0:B*N*N])
for (size_t b=0; b<B; ++b)
 for (size_t i=0; i<N; ++i)
  for (size_t j=0; j<N; ++j) {
   float acc=0.0f;
   for (size_t d=0; d<D; ++d)
    acc += Q[(b*N+i)*D+d]*K[(b*N+j)*D+d];
   S[(b*N+i)*N+j]=acc/sqrtf((float)D);
  }
```

The OpenACC data region and execution configuration are realization obligations; the γ addresses and output ownership are derived from MoA. The displayed code is illustrative and requires compiler, numerical, and runtime validation.

17.4 AST 5c–6c: sequential, OpenMP, OpenACC, and MPI realizations

The sequential ONF is the reference execution. OpenMP assigns disjoint output indices to shared-memory workers; OpenACC assigns them to accelerator workers; MPI partitions the same index domain among processes (with Open MPI as one MPI implementation). For a distributed partition $\mathcal{D}_r$, each process computes only its owned outputs, with required input transfers and any necessary reductions determined by the ψ dependencies. A coordinated OpenMP–OpenACC–MPI execution is a *target architecture*, not an assertion that the current prototype has already generated and validated all four paths. Figure 11 illustrates the intended lowering.

18 Proof obligations and compiler acceptance criteria

A complete mechanization must exhibit a verifiable sequence: (1) shape-correct AST construction; (2) ψ-reduction and DNF semantic preservation; (3) layout-specific γ mapping to sequential ONF; (4) dimension-lifted disjoint coverage and dependency preservation; (5) valid mapping to resource capacities and communication topology; (6) code generation whose numerical semantics match the reference contract; and (7) pre-execution resource, time, and energy predictions assessed against independent hardware measurements. This is a research agenda, not a claim that all obligations have already been discharged by the current prototype.

19 Complete forward attention: DNF to memory-reduced lifted ONF

The previous contraction is only the first part of forward attention. Here we trace the stable normalization and the subsequent value contraction to executable code. This section distinguishes (i) the reference, hand-derived Paper 1 forward equations and their tiled implementation from (ii) the *online-summary realization* developed here as a candidate compiler rewrite. The latter is not attributed to Paper 1's original C implementation. Its correspondence to DNF is established algebraically and checked numerically below.

19.1 AST A: the full mathematical specification

For $\rho Q = \langle n, d\rangle$, $\rho K = \langle m, d\rangle$, $\rho V = \langle m, d_v\rangle$, the result has shape $\rho O = \langle n, d_v\rangle$. The score uses MoA MM with the logical transpose of the last two dimensions of K:

$$s_{ik} = d^{-1/2} \sum_{j=0}^{d-1} (\langle i, j\rangle \psi Q)(\langle k, j\rangle \psi K), \tag{34}$$

$$a_i = \max_{0 \le k < m} s_{ik}, \qquad z_i = \sum_{k=0}^{m-1} \exp(s_{ik} - a_i), \tag{35}$$

$$\langle i, v\rangle \psi O = z_i^{-1} \sum_{k=0}^{m-1} \exp(s_{ik} - a_i)(\langle k, v\rangle \psi V). \tag{36}$$

Figure 12 places the AST rewrite stages alongside their defining formulas. The Cartesian domain $\{(i, k, j, v)\}$ is finite and shape-bounded. The final k-reduction is the MoA value contraction; neither a score matrix nor a probability matrix is required by its denotation.



Figure 12: Traceable full-forward attention ASTs: the score MM transpose-index rewrite, stable normalization, value contraction, stable summary, and addressed/lifted ONF. Each arrow has an explicit equation in the text.

19.2 AST B: stable normalization as a mergeable MoA reduction

For any nonempty key-index subset $H \subseteq \{0, \ldots, m-1\}$, define the summary

$$F_i(H) = (a_H, z_H, \vec{u}_H), \quad a_H = \max_{k \in H} s_{ik}, \quad z_H = \sum_{k \in H} e^{s_{ik} - a_H}, \quad \langle v\rangle \psi \vec{u}_H = \sum_{k \in H} e^{s_{ik} - a_H} (\langle k, v\rangle \psi V). \tag{37}$$

For disjoint H_1, H_2, write $F_i(H_1) \oplus F_i(H_2) = (a, z, \vec{u})$, where

$$a = \max(a_1, a_2), \tag{38}$$

$$z = e^{a_1 - a} z_1 + e^{a_2 - a} z_2, \tag{39}$$

$$\langle v\rangle \psi \vec{u} = e^{a_1 - a}(\langle v\rangle \psi \vec{u}_1) + e^{a_2 - a}(\langle v\rangle \psi \vec{u}_2). \tag{40}$$

Substitution shows that the merged a is the maximum on $H_1 \cup H_2$ and that $z, \vec{u}$ are precisely the sums normalized by this common maximum. Thus the merged summary is equal to $F_i(H_1 \cup H_2)$ in exact arithmetic. The empty set has identity $(a, z, \vec{u}) = (-\infty, 0, \vec{0})$ with the usual guarded update. This establishes a constructive rewrite from the DNF reductions to a fold over k and permits any disjoint partition of the key domain. For floating-point execution, different merge orders may round differently.

19.3 AST C: memory-reduced online ONF

For the ordered key domain $k = 0, \ldots, m-1$, the sequential fold updates $(a, z, \vec{u})$ by merging its current summary with the singleton summary $(s_{ik}, 1, \langle k, \cdot \rangle \psi V)$. Its final output is

$$\langle i, v \rangle \psi O = (\langle v \rangle \psi \vec{u})/z. \tag{41}$$

This realization requires $O(d_v)$ scratch per active query and does not materialize the $n \times m$ score or probability arrays. The original Paper 1 reference C code supplied in the source comparison uses a row scratch array and tiled score operations; the online-summary fold here is a separate, mathematically derived optimization candidate. Neither construction implies zero communication on a key-sharded distributed machine.

19.4 AST D: shape-dependent addresses and instruction lowering

For row-major Q, K, V, O, the selections become

$$\langle i, j \rangle \psi Q = (\text{rav}\, Q)[\gamma_{\text{row}}(\langle i, j \rangle; \rho Q)], \tag{42}$$
$$\langle k, j \rangle \psi K = (\text{rav}\, K)[\gamma_{\text{row}}(\langle k, j \rangle; \rho K)], \tag{43}$$
$$\langle k, v \rangle \psi V = (\text{rav}\, V)[\gamma_{\text{row}}(\langle k, v \rangle; \rho V)]. \tag{44}$$

The corresponding C expressions are `Q[i*d+j]`, `K[k*d+j]`, and `V[k*dv+v]`. The inner j traversals of Q, K are contiguous; $V[k * dv + v]$ traverses the v dimension contiguously for each key. Other γ families yield different regular address sequences, potentially using strided DMA.

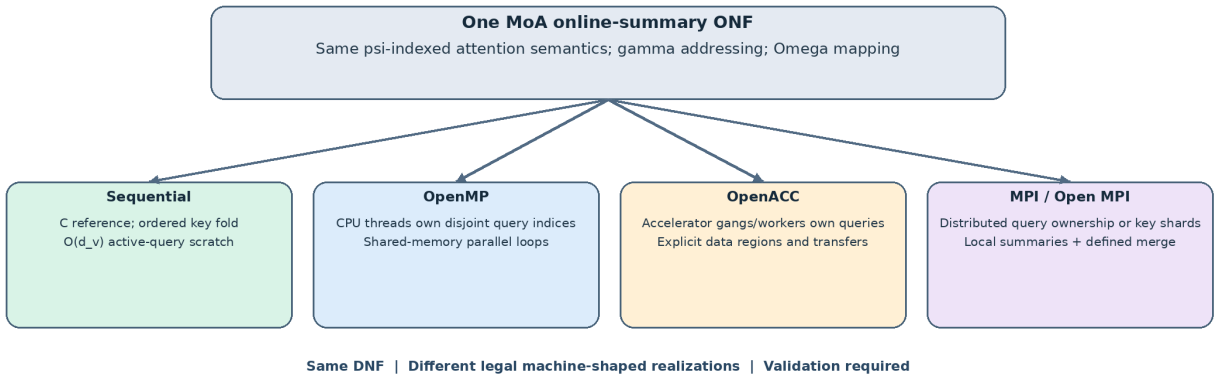


Figure 13: One online-summary ONF and four intended realizations: sequential C, shared-memory OpenMP, accelerator OpenACC, and distributed MPI/Open MPI. A key-sharded distributed mapping merges local $(a, z, \vec{u})$ summaries using Eq. (40). The figure shows the target design rather than verified generation of every backend.

19.5 AST E: CPU, GPU, and coordinated heterogeneous realization

The companion source `attention_full_cpu.c` contains both the DNF reference and a C/OpenMP online-summary realization. The full forward path is shown here, not just the score contraction:

Source listing not bundled in the preserved v38 archive: `attention_full_cpu.c`, lines 37–57. This historical excerpt is intentionally not reconstructed or fabricated.

The CUDA C example `attention_full_gpu.cu` maps one query index to each GPU thread and implements the same summary recurrence. Its demonstrator-local d_v bound is an implementation

constraint, not an MoA restriction. A heterogeneous mapping partitions the query index set into disjoint CPU and GPU subsets, transfers the necessary K, V data, launches the GPU subset, and combines the disjoint output regions. The companion CUDA source includes the GPU kernel and a launch fragment. It is *illustrative and not yet validated on a CUDA device*; a production realization must include transfer, synchronization, error handling, and machine-dependent scheduling.

19.6 Validation and compiler acceptance gate

For $n = 7, m = 11, d = 9, d_v = 5$ and deterministic inputs, the CPU DNF reference and OpenMP online realization were compiled with GCC and compared: maximum absolute output difference 1.11×10^{-16}. This is a small numerical smoke test, not a general proof of IEEE floating-point equivalence or GPU correctness. The mathematical summary identity supplies the exact-arithmetic equivalence argument. The compiler acceptance gate is an emitted AST rewrite trace, proof of partition coverage, shape-correct γ addresses, a faithful target lowering, and numerical tests over varying shapes, layouts, and target resources. Time and energy optimality remain empirical and predictive research objectives.

20 A reproducible AST-to-C mechanization prototype

The previous code examples were hand-written realizations. This section adds a *small executable code-generation prototype* rather than claiming that a general MoA compiler is complete. The companion script `moa_ast_codegen.py` records five structured AST stages (A–E) as `ast_rewrite_trace.json`, emits `generated_attention_cpu.c`, and checks bounded index properties. The AST stages are: (A) attention specification, (B) ψ-reduced DNF, (C) stable $(a, z, \vec{u})$ summary fold, (D) shape-dependent γ addressing, and (E) query-domain ownership under Ω lifting. The source-to-source generator is presently a restricted, template-based lowering of this single derived operation; it does not yet parse arbitrary Fortran, MATLAB, Python, or Julia expressions.

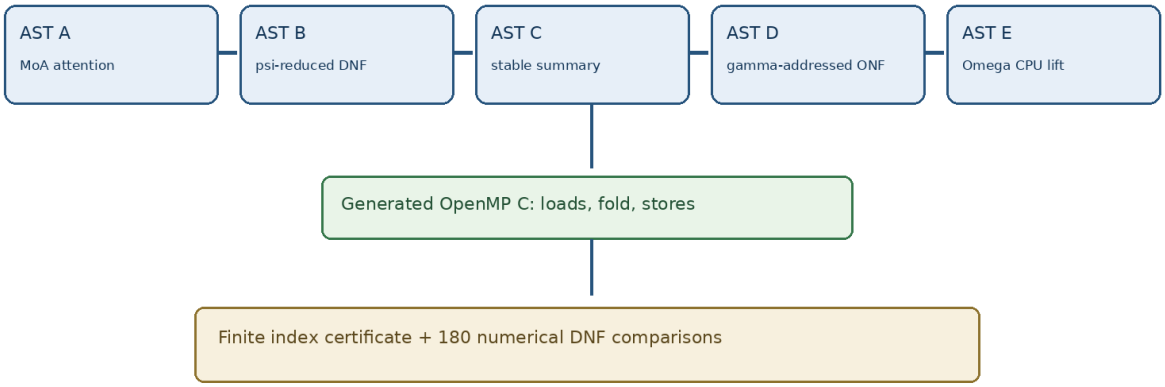


Figure 14: Executable prototype trace: each named MoA AST stage precedes CPU code emission. The bounded certificate verifies address and ownership properties on sampled finite shape domains; the exact-arithmetic summary identity is established separately in Eq. (40).

Index and ownership certificate. For row-major arrays, $\gamma_{\text{row}}(\langle i, j\rangle; \langle r, c\rangle) = ic + j$ on $0 \le i < r$, $0 \le j < c$. The generator enumerates 288 finite combinations of positive n, m, d, d_v and worker counts, checking that this γ is a bijection onto $[0, rc)$ for each relevant array shape and that the lifted ownership rule $i = w + tp$ $(0 \le w < p,\ t \ge 0,\ i < n)$ assigns every query index exactly once. These exhaustive

checks *within the declared finite domain* are reproducible evidence, not a proof for all natural-number shapes. For arbitrary positive p, uniqueness follows because $i \bmod p$ identifies the unique owner w, and completeness follows by Euclidean division $i = p\lfloor i/p \rfloor + (i \bmod p)$.

Generated instructions and test. The generated C uses the indexed contraction $Q[\gamma(\langle i, j\rangle; \rho Q)]K[\gamma(\langle k, j\rangle; \rho K)]$, then the stable summary recurrence, with one OpenMP-owned output row per i. The emitted kernel is in `generated_attention_cpu.c`; `test_generated.c` compares it to a separately coded materialized-score DNF reference. With GCC `-O2 -fopenmp`, 180 positive-shape cases passed; the maximum observed absolute difference was 4.44×10^{-16}. This supports a limited numerical regression claim, not bitwise equivalence or formal verification of generated instructions.

Target scope and next proof gates. The CUDA and CPU–GPU paths were subsequently compiled and exercised on UAlbany DGX hardware, as documented in Section 22; this does not establish arbitrary-shape correctness or an optimal scheduler. To reach the universal objective, the generator must accept general MoA ASTs, implement verified rewrite rules rather than fixed templates, emit per-step certificates, and lower the same lifted ONF into CPU, GPU, and coordinated CPU–GPU instruction sequences with validated transfer and synchronization behavior. Performance, power, and energy predictions remain separate empirical obligations.

21 Restricted rewrite passes and three executable target lowerings (v12)

We extend the earlier fixed CPU template with an explicit sequence of *typed, restricted* AST passes, implemented in `moa_rules_v12.py`. The input is a forward-attention node with shapes $\rho Q = \langle n, d\rangle$, $\rho K = \langle m, d\rangle$, $\rho V = \langle m, d_v\rangle$, and $\rho O = \langle n, d_v\rangle$. The passes are not a general parser or a formal proof assistant. Their observable intermediate nodes are recorded in `ast_rule_trace_v12.json`:

$$\underbrace{\mathrm{MM}(Q, K^{T_{\mathrm{last}\,2}})}_{\mathrm{AST}_0} \xrightarrow{\psi\text{-composition}} \underbrace{\sum_j (\langle i, j\rangle \psi Q)(\langle k, j\rangle \psi K)}_{\mathrm{AST}_1} \xrightarrow{\text{stable summary}} \underbrace{(a, z, \vec{u})}_{\mathrm{AST}_2} \xrightarrow{\gamma(\vec{\imath}; \rho X)} \mathrm{AST}_3 \xrightarrow{\Omega_{\mathcal{M}}} \mathrm{AST}_4. \tag{45}$$

The first AST arrow is the mathematical transpose-index composition; the second is the exact-arithmetic summary identity in Eq. (40). The addressing pass uses the shape of *each selected array*, rather than a shape-free offset. The final ownership pass is parameterized by CPU, GPU, or coordinated CPU–GPU target.

CPU instantiation. The generated `generated_v12_cpu.c` implements the stable summary using OpenMP query ownership and the shape-derived row-major γ address family. Each query index is assigned once, and the j-reduction visits contiguous components of the original Q and K. The separate materialized-score DNF reference and the generated CPU realization were compared across 180 positive-shape cases, with a maximum absolute difference of 4.44×10^{-16} in the recorded v12 regression. This is numerical evidence, not a proof of identical IEEE floating-point evaluation.

GPU instantiation. The generated `generated_v12_gpu.cu` maps query indices to CUDA block-/thread coordinates. The same γ operands and stable summary recurrence appear inside each thread. The demonstrator uses per-thread local scratch of at most 64 value components; this is an implementation bound, not an MoA restriction. Subsequent UAlbany DGX compilation and numerical testing are reported in Section 22.

Coordinated CPU–GPU instantiation. The generated `generated_v12_cpu_gpu.cu` partitions the query domain into $[0, s)$ for CPU and $[s, n)$ for GPU. It includes device allocation, input transfers, a GPU launch, concurrent host computation, synchronization, transfer of the GPU-owned output range, and cleanup. The sets are disjoint and cover $[0, n)$, but optimal s is not yet predicted. In particular, this host coordinator is generated as a target-specific wrapper around the shared lifted operation, not as a proved optimal heterogeneous scheduler.

Evidence and limitations. The current construction demonstrates reusable named transformation passes and three backend code emitters for a *restricted attention operation*. The AST trace is inspectable, but no independent machine-checkable proof certificates have been produced. The restricted CPU, CUDA, and coordinated CPU–GPU paths have now been compiled and numerically exercised on DGX hardware (Section 22); generality, formal proof certificates, and predictive timing remain open. General source-language front ends, multiple layouts, cross-device summary reductions, and prediction of time and energy from deterministic machine shapes remain future compiler milestones.

22 DGX hardware validation of generated CPU–GPU attention (v13)

Provenance and claim boundary. This section reports an additional October 30, 2026 experimental session on University at Albany DGX nodes, distinct from the earlier calibration/holdout study. The restricted v12 code generator emitted the CPU, CUDA, and coordinated host wrapper. The source and measurement transcript supplied for this update establish execution and finite numerical tests; they do not establish a proof of unrestricted machine-shape correctness, optimal splitting, or a calibrated time/energy predictor. Prior attention studies were hand-derived and implemented, not produced by this compiler.

22.1 MoA output ownership and execution realization

For the tested forward-attention output with $\rho O = \langle n, d_v \rangle$, the CPU–GPU split s assigns query-index sets

$$I_{\mathrm{CPU}} = \{i \mid 0 \le i < s\}, \quad I_{\mathrm{GPU}} = \{i \mid s \le i < n\}, \quad I_{\mathrm{CPU}} \dot{\cup} I_{\mathrm{GPU}} = \{0, \ldots, n-1\}. \tag{46}$$

Each output $\langle i, v \rangle \psi O$ is owned exactly once. The tested coordinator allocates device buffers, transfers Q, K, V, launches GPU work for I_{GPU}, executes the CPU-owned ψ-indexed work, synchronizes, transfers GPU-owned output components, and releases device buffers. The index-to-address realization uses γ on the original shaped arrays; the split changes ownership, not the DNF.

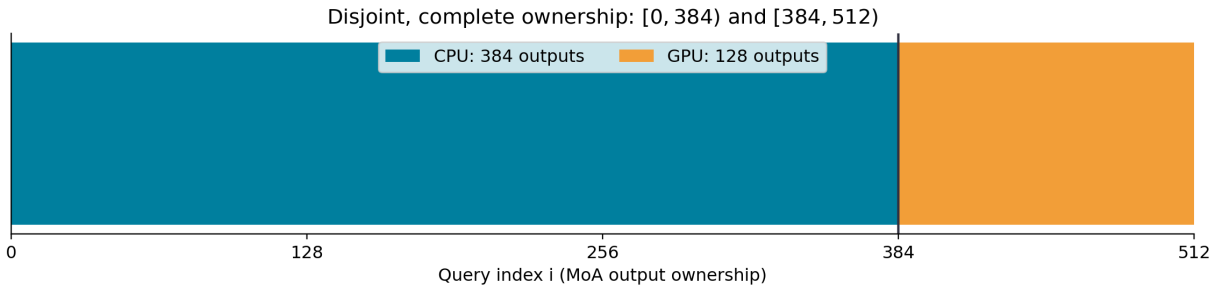


Figure 15: Color-coded, complete ownership for $n = 512$, $s = 384$. The CPU owns 75% of query indices; the GPU owns the remaining 25%.

22.2 Experimental configuration and numerical checks

The measured attention shape was $n = m = 512$, $d = d_v = 64$, with four OpenMP threads bound to cores $0, 1, 64, 65$ on DGX12, an AMD EPYC 7742 node with an NVIDIA A100-SXM4-80GB GPU. The diagnostic used one untimed warmup and ten timed wrapper invocations, with `CLOCK_MONOTONIC` timing. CUDA 12.8 and Nsight Systems 2024.6.2 were used for kernel and runtime profiling. The CPU ownership split was $s = 384$; the GPU processed 128 query indices. A checksum of 125.183536304 was reproduced by the phase-instrumented benchmark.

The reported regression sessions also tested five ownership splits (GPU-only, CPU-25%, CPU-50%, CPU-75%, and CPU-only) on 36 larger shapes each, with observed maximum absolute numerical differences no larger than 1.84×10^{-16} in those runs. Earlier tests covered 180 shape cases per mapping. These are finite numerical comparisons, not a proof of bitwise-identical floating-point results for all shapes.

22.3 Kernel stability versus host-wrapper variability

Nsight Systems recorded 11 executions of `moa_gpu_rows`, including warmup: mean 6.707529 ms, median 6.707384 ms, minimum 6.703128 ms, maximum 6.710808 ms. The full range was only 7.680 microseconds. GPU launch-to-execution delays were typically 13–14 microseconds. Figure 16 displays each kernel duration.

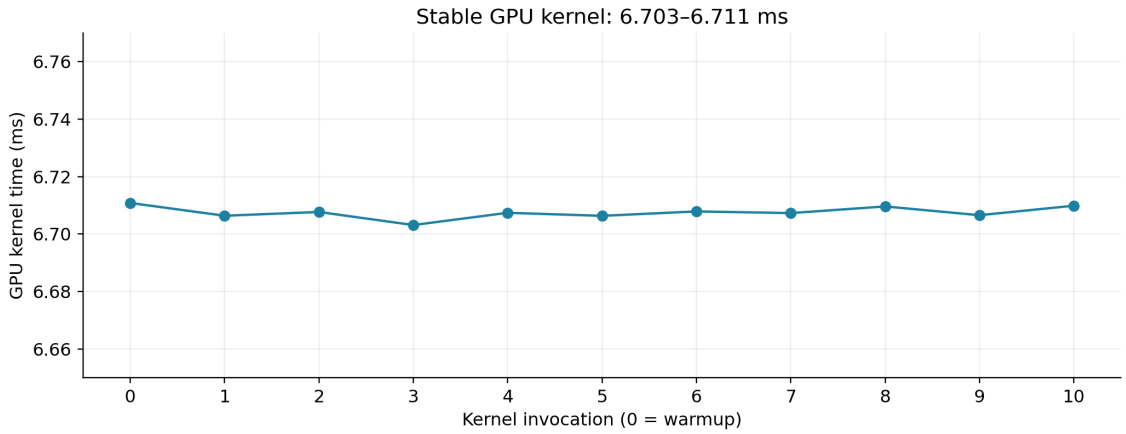


Figure 16: GPU kernel timing on DGX12 for the CPU-75% mapping. The GPU computation was stable despite variability in the complete host wrapper.

For the same mapping, the phase-instrumented benchmark recorded ten wrapper invocations with mean 7.473673 ms, minimum 7.270481 ms, and maximum 8.601487 ms. The measured CPU-owned range typically took about 5.53 ms (timed range 5.513719–5.707848 ms). The CPU and GPU durations overlap and must *not* be added to predict total elapsed time. Figure 17 compares the independently observed quantities without implying additivity.

22.4 Measured phase costs and outliers

The wrapper was instrumented around CUDA allocation, host-to-device transfers, the CPU-owned work, device synchronization, device-to-host output transfer, and buffer cleanup. The measured phase costs are *host-side elapsed intervals*; CUDA synchronization is a wait for outstanding GPU work, not an independent GPU computation cost. The GPU launch and small bookkeeping costs are included in the total but not represented as a separate stacked phase in Figure 18.

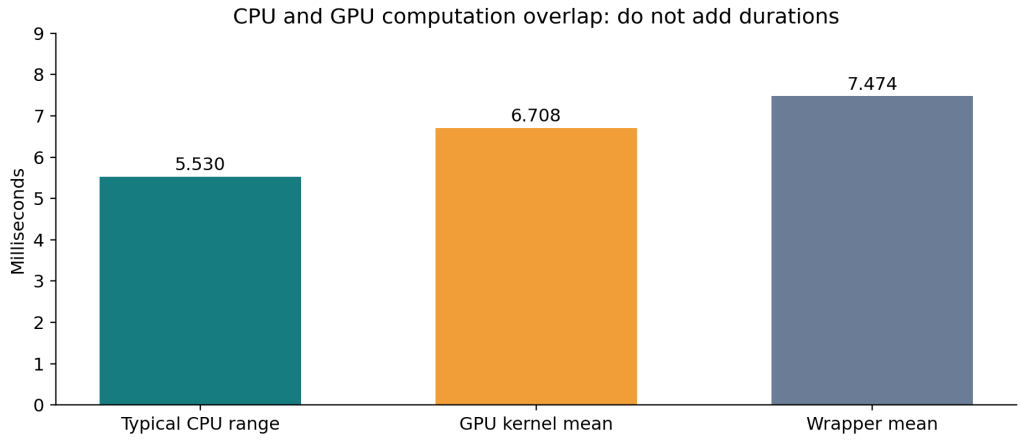


Figure 17: CPU-owned computation, GPU kernel, and end-to-end wrapper times are different measurements. CPU and GPU work execute concurrently.

Table 4: Representative phase durations in the ten timed DGX12 CPU-75% invocations. Values are approximate typical durations, except where identified as observed outliers.

Phase	Typical time (ms)	Observed variation
CUDA allocation	0.21	1.536 in repeat 6
Host-to-device transfer	0.105	relatively stable
CPU-owned computation	5.53	5.514–5.708
CUDA synchronization	1.20	1.031–1.218
Device-to-host output transfer	0.022	relatively stable
CUDA cleanup	0.19–0.20	0.685 in repeat 5

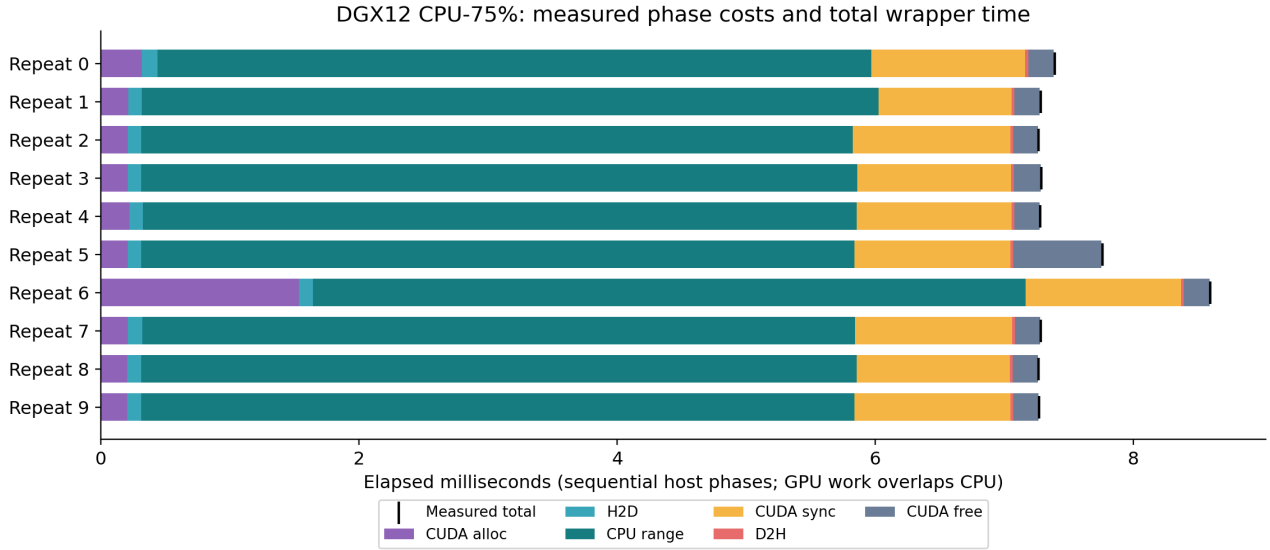


Figure 18: Color-coded measured host phase intervals across ten timed invocations; black ticks mark total wrapper times. Phase sums omit small launch/bookkeeping intervals. Repeat 5 exhibits longer cleanup; repeat 6 exhibits longer allocation. GPU kernel execution overlaps CPU work.

Repeat 5 took 7.765451 ms, with cleanup measured at 0.684628 ms. Repeat 6 took 8.601487 ms, with allocation measured at 1.536014 ms. The untimed warmup took 434.711314 ms in this phase-instrumented run, including 342.016789 ms in the allocation phase; warmup was excluded from the reported mean. A separate Nsight trace recorded a 194.855754 ms first allocation, also during warmup. Different startup measurements must not be conflated.

In the Nsight trace, one invocation entered `cudaDeviceSynchronize` approximately 1.265 ms *after* the GPU kernel had finished, producing a roughly 0.005 ms synchronization call. The generated source calls `moa_cpu_range` before synchronization, so this indicates delayed host progress on that invocation, but does not isolate arithmetic from OpenMP scheduling or other host overhead. An earlier timed invocation of approximately 408 ms was observed in another run; its cause was not established and it was not reproduced in the final phase-level experiment.

22.5 Implications for machine-shape prediction

The finite numerical tests support the correctness of the tested Ω-lifted ownership realization. The GPU kernel timings show a stable computation component. The host wrapper, however, includes allocation, data movement, scheduling, synchronization, and cleanup that can vary independently of the mathematical index partition. A useful end-to-end model must describe the temporal overlap and the runtime phases without double counting:

$$T_{\text{wrapper}} = T_{\text{allocation}} + T_{\text{H2D}} + T_{\text{launch/host}} + T_{\text{overlap interval}} + T_{\text{D2H}} + T_{\text{cleanup}}, \tag{47}$$

where the overlap interval is determined by host progress and GPU completion, not by the sum of their compute durations. This expression is an accounting structure, *not* a validated exact prediction formula. Earlier GPU-board power measurements are limited to the GPU board and cannot establish total CPU–GPU system energy. These experiments therefore validate finite mappings and expose measurable prediction obligations, rather than close the prediction gap for arbitrary hardware.

23 Portable realization: OpenMP, OpenACC, and MPI

Backend provenance and portable direction. The v13 AST sequence and its rewrite semantics remain unchanged. In v12–v13, the generated accelerator source is *CUDA* (`generated_v12_gpu.cu`), and the coordinated CPU–GPU wrapper is also CUDA (`generated_v12_cpu_gpu.cu`); neither should be relabeled OpenACC. The CPU backend used C and OpenMP. The subsequent development adds an OpenACC realization and extends OpenMP work. These are additional backend realizations of the same MoA-derived indexed computation, not corrections to the v13 ASTs. The supplied v13 project contains the CUDA and C/OpenMP sources but does not contain the new OpenACC generator, emitted OpenACC source, or its execution logs. Accordingly, this version records the new backend direction without asserting independently reproduced OpenACC compilation, numerical comparisons, or measured speedups. Those files and tests are a required v14 reproducibility addition.

Target-independent versus target-specific obligations. For each realization $b \in \{\text{OpenMP}, \text{OpenACC}, \text{MPI}\}$, the source of truth is a common ψ-indexed DNF and the γ-addressed, Ω-lifted ONF. The backend must demonstrate that (i) every legal output index has exactly one owner or a specified reduction, (ii) all indexed reads refer to the intended logical component, (iii) device data lifetimes and transfers are correct, and (iv) synchronization preserves dependencies. OpenMP and OpenACC directives and MPI message passing realize MoA-derived mappings; they are not alternative mathematical foundations. MPI communication must also preserve the specified ownership, dependencies, message payload, and synchronization conditions. A portable source representation does

not by itself establish portable performance. A finite numerical regression supports tested configurations only; compiler-generated code should not be credited with earlier hand-derived attention results.

24 A shaped communication extension: deterministic routing and port scheduling (Steps 68–73)

Logical machine shape and routed broadcast. Let the logical machine have shape $M = \langle m_0, \ldots, m_{r-1} \rangle$ with $m_j > 0$ and $P = \tau(M) = \prod_j m_j$. Coordinates are members of $\iota(M)$. The root is $\langle 0, \ldots, 0 \rangle$. For any nonroot coordinate c, define parent(c) by decrementing its last nonzero coordinate by one. This defines a rooted nearest-neighbor spanning tree with $P - 1$ directed edges. Each nonroot coordinate has exactly one parent and repeated parent applications reach the root, so no directed cycles exist. The construction is an ideal logical topology; it is not a measured representation of DGX network links.

Port-constrained stages. Given an outgoing-port cap $q \geq 1$, a deterministic greedy scheduler assigns tree edges to stages. A parent may send along at most q edges in one stage, and a recipient may forward only in a *subsequent* stage. A separate verifier checks exact edge coverage, one reception per nonroot coordinate, parent readiness, port caps, and total logical traffic. This is a legal schedule for the stated model, not a proof of minimum possible stage count.

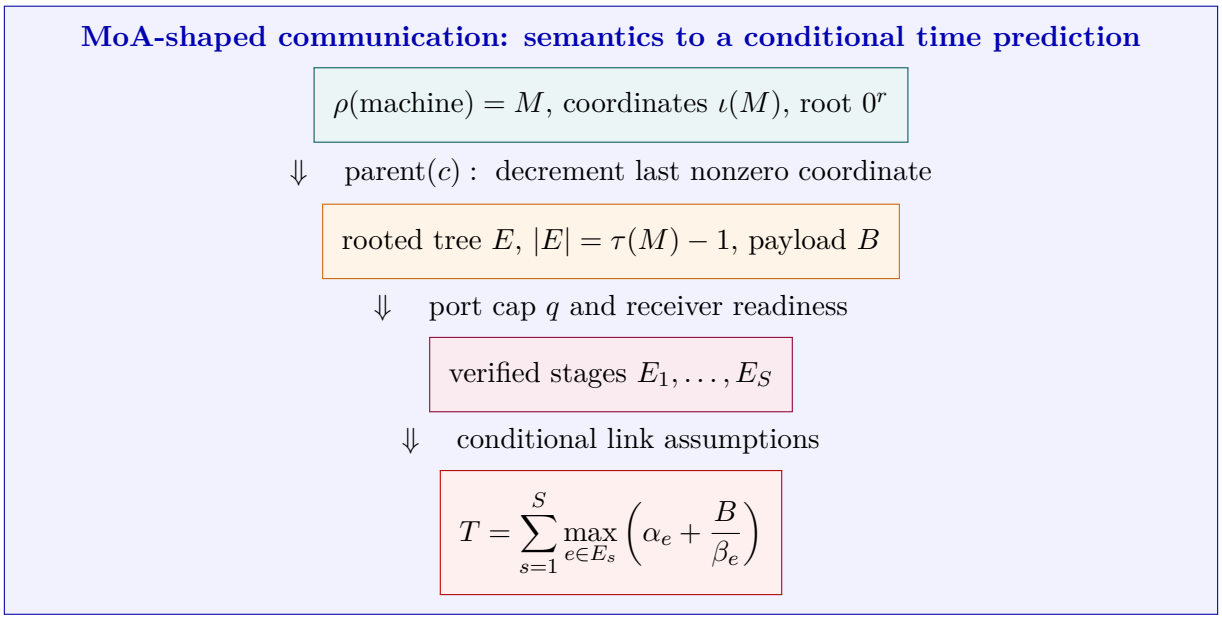


Figure 19: The new v14 communication branch extends machine-shape realization without altering the attention AST or DNF. Every arrow has a separately testable obligation. The final timing formula assumes concurrent transfers within each stage and no unmodeled contention; it is not a measured hardware model.

Verified finite test matrix. For a 440-byte broadcast payload, the following stage counts were obtained by the fixed greedy schedule. Logical link volume is $(P - 1)B$, independent of the port cap.

Machine shape M	P	$q=1$	$q=2$	$q=100$	Link bytes
$\langle 1\rangle$	1	0	0	0	0
$\langle 2\rangle$	2	1	1	1	440
$\langle 2,3\rangle$	6	4	3	3	2200
$\langle 3,2\rangle$	6	5	3	3	2200
$\langle 2,2,3\rangle$	12	7	5	4	4840

The tests covered 15 shape/port combinations and included rejection checks for invalid communication parameters. Shape order affects the constructed tree and schedule even when the number of logical coordinates is equal. These are finite tests of a specified implementation, not a general formal proof of scheduler correctness.

Conditional communication-time models. For uniform link latency α and bandwidth β, with every stage using at least one edge, the model reduces to

$$T_{\text{uniform}} = S\left(\alpha + \frac{B}{\beta}\right). \tag{48}$$

The heterogeneous extension assigns each directed edge e its own α_e and β_e and predicts

$$T_{\text{hetero}} = \sum_{s=1}^{S} \max_{e\in E_s}\left(\alpha_e + \frac{B}{\beta_e}\right). \tag{49}$$

For $M=\langle 2,3\rangle$, $q=2$, and $B=440$ bytes, illustrative uniform parameters $\alpha = 2\,\mu$s and $\beta = 10\,\text{GB/s}$ give $6.132\,\mu$s across three stages. Replacing the link from $\langle 0,0\rangle$ to $\langle 0,1\rangle$ with $\alpha_e = 5\,\mu$s and $\beta_e = 2\,\text{GB/s}$ gives $9.308\,\mu$s; the stage costs are 5.220, 2.044, and $2.044\,\mu$s. The heterogeneous implementation also reduces to the uniform prediction when all link parameters coincide. These numbers are *illustrative analytic outputs*, not observed DGX timings. The model omits contention, link-sharing, protocol overhead, and physical topology calibration.

Reproducibility and next compiler obligation. The Steps 68–73 scripts and test transcripts are not included in the supplied v13 source archive; this section documents the verified development checkpoint but cannot substitute for those source artifacts. A submission-quality package must include the routing, scheduling, verification, and timing scripts plus machine-readable test outputs. Cost-aware scheduling (Step 74), calibrated physical link parameters, OpenACC regression evidence, and automatic reproduction of the hand-derived attention implementations remain future work. In particular, selecting a schedule after observing its cost is distinct from proving that the selected schedule minimizes time.

25 November 2026 compiler integration: typed ASTs, verified ownership, and audited topology

This section updates the preserved v16 research manuscript with the concrete `research/typed_attention/moa_typed_ir.py` source and the November 3 DGX test transcript. Earlier sections and their historical experimental evidence are retained for traceability. In particular, historical accelerator prototypes must not be confused with the current portable OpenACC direction. This is a *restricted forward-attention mechanization*, not yet a universal MoA compiler.

25.1 The actual typed semantic AST, not a hypothetical compiler tree

The immutable Python dataclasses are `Array(name,shape)`, `Psi(array,indices)`, `Product(lhs,rhs)`, `Reduce(index,term)`, `Scale(term,factor)`, and `SoftmaxWeightedReduce(score,values,key_index,value_index,stable)`. For $\rho Q = \langle n, d\rangle$, $\rho K = \langle m, d\rangle$, $\rho V = \langle m, d_v\rangle$, the actual tree from `specification()` is rooted at `SoftmaxWeightedReduce`; its score child is a `Scale` of a j-`Reduce` over a `Product` of $\langle i, j\rangle\psi Q$ and $\langle k, j\rangle\psi K$, while its value child is $\langle k, v\rangle\psi V$. The scale factor is $1/\sqrt{d}$. The key and value index fields are respectively k and v. The JSON field named `dnf` is the *current restricted semantic representation*; the supplied implementation does not establish a general, complete ψ-normalization procedure.

25.2 Checked stable online summary and numerical contract

`stable_online_fold()` accepts only the exact supported ψ dependencies and stable scaled reduction. It produces `OnlineSummary(score,values,key_index,value_index,state)`, with state $\langle a, z, u\rangle$. For one query i and keys processed through k, the online summary has maximum a, scaled normalizer z, and value accumulator u; its output is u/z. The concrete generated source uses, for each next score s and value component v,

$$a' = \max(a, s), \qquad h = \begin{cases} 0 & z = 0 \\ e^{a-a'} & z \neq 0 \end{cases}, \qquad f = e^{s-a'}, \tag{50}$$

$$z' = hz + f, \qquad u'_v = hu_v + f(\langle k, v\rangle\psi V), \qquad (\langle i, v\rangle\psi O) = u_v/z. \tag{51}$$

The equivalence claim is exact-arithmetic algebra under defined nonempty extents. Floating-point order, exceptional values, and the backend's numerical tolerance remain explicit acceptance conditions.

25.3 Actual γ-addressed ONF AST

`gamma_lower()` maps the checked summary to `LoweredAttention(summary,addresses,ownership)`, with ownership `query-disjoint`. Each address is a `GammaAddress` AST node:

array	ψ selection	current row-major address
Q	$\langle i, j\rangle\psi Q$	$(\mathrm{rav}\, Q)[id + j]$
K	$\langle k, j\rangle\psi K$	$(\mathrm{rav}\, K)[kd + j]$
V	$\langle k, v\rangle\psi V$	$(\mathrm{rav}\, V)[kd_v + v]$
O	$\langle i, v\rangle\psi O$	$(\mathrm{rav}\, O)[id_v + v]$

(52)

The address mapping is one member of the MoA γ family; it is not a claim that row-major addressing is the only possible layout. The present `verify()` checks the expected four address triples and ownership string. It does not prove correctness for arbitrary layouts.

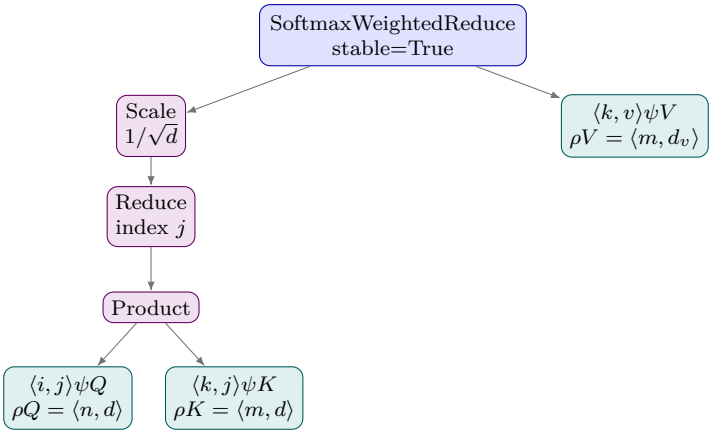


Figure 20: **Actual restricted semantic AST** reconstructed from the supplied `specification()` function and `typed_derivation_trace.json`. Blue denotes the attention root, purple denotes operations, and teal denotes ψ-indexed leaves. Field labels k, v and the stability flag belong to the root, rather than additional invented rewrite nodes.

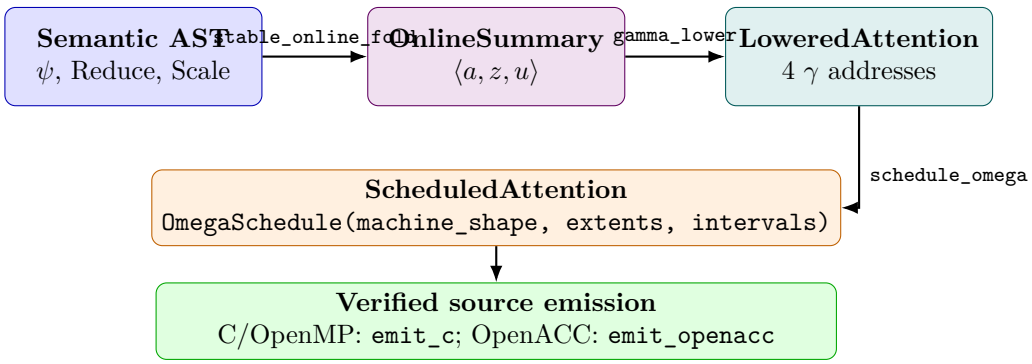


Figure 21: The current concrete AST and source-emission chain. The source supports restricted forward attention and verifies the supported patterns; this is not a claim of universal automatic normalization, optimal scheduling, or measured accelerator execution.

25.4 Ω ownership, resource constraints, and independent routing audit

`OmegaSchedule` stores machine shape, query/key/score/value extents, and ownership intervals; `ScheduledAttention` pairs that schedule with the lowered AST. The basic `schedule_omega()` delegates to `omega_lift()` and checks interval coverage. A distinct, additive resource-constrained planning layer `ResourceOmegaPlan` permits heterogeneous query counts and idle machine components; its ownership is then consumed by the communication certificate and topology constructor. The abstract machine is a shaped array, not an MPI implementation choice.

For a shaped machine M with $P = \tau(M)$ components, an active owner p with query interval $[b_p, e_p)$ has payload

$$B_p = (e_p - b_p)d_v w, \qquad L_p = (P-1)B_p, \qquad L_{\text{total}} = \sum_{p:B_p>0} L_p, \tag{53}$$

where w is bytes per value component. The current candidate realization uses a deterministic owner-rooted nearest-neighbor spanning tree on the logical machine shape. For each nonempty owner, the route has $P-1$ directed edges, and its audited link-byte volume agrees with the logical nonlocal demand. This equality is *not* a measurement of physical DGX link traffic and does not imply that MPI_Allgatherv uses this tree.

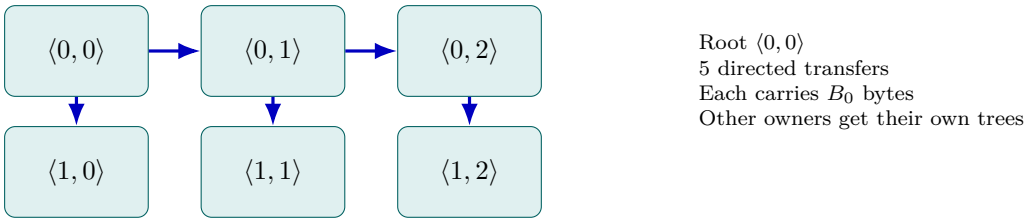


Figure 22: Illustrative logical $\rho M = \langle 2, 3\rangle$ owner-rooted nearest-neighbor tree for the first owner. Arrows show a valid directed spanning tree on the abstract machine; the independent auditor checks adjacency, receive-before-forward, unique delivery, reachability, hop count, and traffic accounting.

The independent `audit_omega_topology()` verifies actual owner coordinates and intervals against the resource plan, correct component and byte counts, nearest-neighbor transfers, absence of duplicate edges, receive-before-forward order, complete reachability, maximum hops, and aggregate traffic. It is intentionally stricter than the topology object's internal `verify()` method. The topology layer is currently an *additive research module*; integration into a general AST lowering pipeline and physical timing model is still work to be done.

25.5 Observed regression evidence and outstanding acceptance gates

On November 3, 2026, the DGX terminal transcript reported `Ran 24 tests in 0.033s -- OK` for `test_attention_omega_topology.py`. These included valid heterogeneous and idle ownership, a singleton machine, multidimensional machine shapes, and deliberate corruptions of routes, dependencies, intervals, element widths, hop counts, and traffic totals. In particular, `test_self_consistent_wrong_ownership` passed: consistent aggregate accounting did not mask incorrect ownership. This is a reported unit-test outcome, *not* a hardware network performance measurement. An earlier full-suite baseline of 123 tests preceded the topology modules. Subsequent verified milestones increased the suite to 170 passing tests after port scheduling, 202 after heterogeneous communication-time prediction, 218 after directed-link contention prediction, and 234 after candidate schedule selection (see Sections 26 and 27).

Current evidence	Still required for SC27 acceptance
Typed forward-attention AST, restricted checked summary rule, four γ addresses	General ψ-reduction/normalization and broader semantic AST coverage
C/OpenMP and OpenACC source emitters in inspected Python source	Independent compilation, execution, and numerical validation of all emitted backends on target systems
Basic and resource-constrained Ω ownership, communication certificates and topology auditor	Integrate all certificates into the compiler pass trace and establish composition of correctness obligations
234 passing compiler tests on DGX (latest checkpoint), including independent contention and schedule-selection tests	Prospective calibrated time/energy predictions, physical hardware measurements, and exhaustive small-case optimality certificates
Four hand-derived attention/Transformer studies as targets	Compiler reproduction of forward, backward, decode, and Transformer-block implementations and their predictions

Reproducibility and source boundary. The source text of `moa_typed_ir.py` and the JSON derivation trace were supplied verbatim in the research conversation; the newer topology modules and test results were supplied during the preceding DGX collaboration. The current working repository is `https://github.com/womenflyplanes/moa-compiler`. The manuscript does not assert that all current DGX files were independently retrieved or that a post-November-3 commit was pushed. The November 3 local commit identifiers for the topology/scheduling, heterogeneous timing, contention prediction, and candidate selection milestones are `f994221`, `c62d214`, `faf06be`, and `018bc83`, respectively; none is claimed here to have been pushed to a remote. Archive the full test transcript and exact repository state before submission.

26 Verified November compiler extension: Ω port scheduling and heterogeneous communication time

This section reports the subsequent November 3, 2026 DGX-head-node compiler development checkpoint. It extends the typed forward-attention resource-constrained Ω ownership and topology described in Section 25; it does not alter the earlier hand-derived attention implementations or establish end-to-end MPI execution. The two local commits are `f994221` (audited topology and port scheduling) and `c62d214` (heterogeneous time model, independent auditor, and tests), on branch `develop/mac-compiler-integration`.

26.1 From Ω ownership to legal port-constrained stages

Let $M = \rho(\text{machine})$ and let the resource plan assign each active owner $p \in \iota(M)$ an output interval $[b_p, e_p)$, with payload $B_p = (e_p - b_p)d_v w$ bytes. The topology constructor produces an owner-rooted nearest-neighbor spanning tree on the abstract shaped machine for each nonempty owner. The route is a *candidate logical realization*, not a physical network topology or a statement about the implementation of `MPI_Allgatherv`. For each owner, its route has $\tau(M) - 1$ edges; hence the logical aggregate link volume is

$$L_{\text{logical}} = \sum_{p:B_p>0} (\tau(M) - 1)B_p. \tag{54}$$

A deterministic greedy scheduler partitions the transfers into ordered stages $S_0, \ldots, S_{k-1}$. In any stage, each parent initiates at most q outgoing transfers; a receiver may forward a payload only in a *subsequent* stage. The independent schedule auditor reconstructs expected transfers from the ownership plan and topology, then checks exact transfer coverage, per-parent port limits, receive-before-forward, reachability, machine shape, stage accounting, and logical link bytes. The resulting schedule is legal for the specified abstract constraints; minimal stage count has not been proved.

26.2 A conditional time functional over the verified stages

For each scheduled transfer e carrying B_e bytes on directed link $\ell(e)$, assign finite nonnegative latency $\alpha_{\ell(e)}$ and finite strictly positive effective bandwidth $\beta_{\ell(e)}$. The transfer-time model is

$$t_e = \alpha_{\ell(e)} + \frac{B_e}{\beta_{\ell(e)}}. \tag{55}$$

Under the explicit assumptions that transfers in one stage execute concurrently without additional contention, and stages execute in sequence, the conditional prediction is

$$\boxed{T_{\text{comm}} = \sum_{s=0}^{k-1} \max_{e \in S_s} \left(\alpha_{\ell(e)} + \frac{B_e}{\beta_{\ell(e)}} \right).} \tag{56}$$

An empty stage contributes zero, and a singleton machine with no communication has $T_{\text{comm}} = 0$. The uniform-link fallback assigns the same (α, β) to every required directed link. Unlike the earlier fixed-payload broadcast expression in Equation (49), Equation (56) allows owner-dependent payload sizes. It is not a calibrated physical network model: incoming-port limits, simultaneous use of shared links, protocol costs, congestion, and computation–communication overlap are not yet represented.

26.3 DGX-head-node regression evidence

The complete `tests/universal_ir` suite reported `Ran 202 tests in 1.885s -- OK` on `dgx-head01`. The dedicated timing module reported 32 passing tests in 0.073 seconds, including 7 ordinary cases, 11

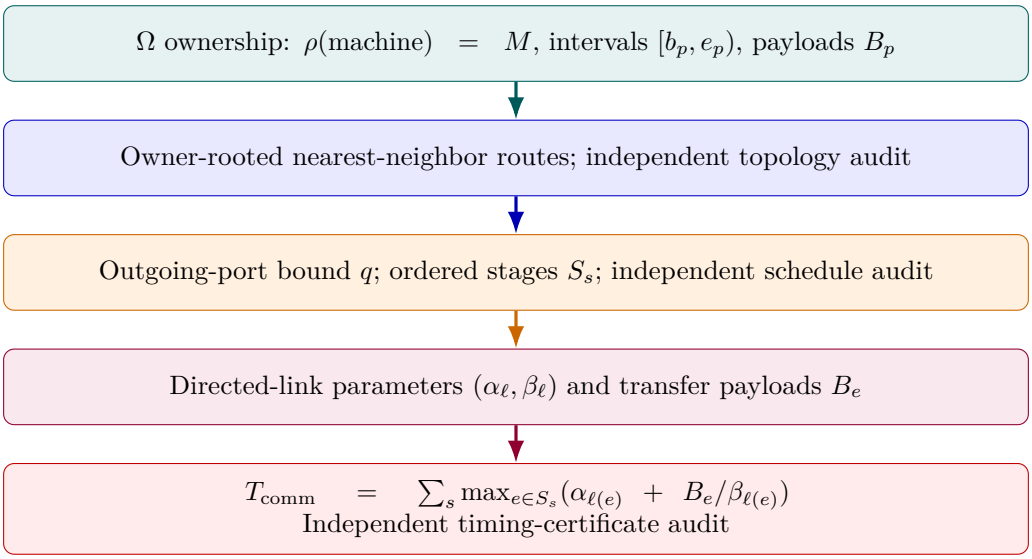


Figure 23: Verified MoA machine-shape communication derivation in the November compiler prototype. Each stage has independently checkable ownership, transfer, or timing obligations. The final time is conditional on the stated stage-concurrency assumptions, not measured hardware time.

invalid-parameter cases, and 14 adversarial certificate mutations. An independently constructed auditor checks the original ownership and schedule, exact link-key coverage, finite parameter domains, each stage maximum, and total time. It rejects self-consistent but incorrect stage and total times. These are reported software tests on the head node, not network timing experiments.

Table 5: Verified logical schedules for heterogeneous Ω ownership. The stages are deterministic greedy outputs, not proven optima.

Machine shape and ownership	$q = 1$	$q = 2$	$q = 100$	Link bytes
$\langle 2, 3\rangle$, counts $(1, 2, 1, 1, 1, 1)$	11	6	3	840
$\langle 2, 3\rangle$, counts $(3, 0, 0, 0, 0, 0)$	3	3	3	480
$\langle 2, 2, 2\rangle$, all counts 1	12	6	3	896
$\langle 1\rangle$, count 4	0	0	0	0

For $M = \langle 2, 3\rangle$, counts $(1, 2, 1, 1, 1, 1)$, $q = 1$, and $L_{\text{logical}} = 840$ bytes, the uniform-link inputs $\alpha = 2\,\mu$s and $\beta = 10\,$GB/s yield $T_{\text{comm}} = 22.036\,\mu$s across 11 stages. Replacing the parameters on directed link $\langle 0, 0\rangle \to \langle 0, 1\rangle$ with $\alpha = 8\,\mu$s and $\beta = 1\,$GB/s yields $34.0744\,\mu$s with the *same* ownership, stages, and 840 logical link bytes. The predicted increase is approximately 54.6%. Both certificates passed the independent audit. These are analytic outputs of an assumed link model, not DGX latency observations.

Acceptance boundary and next step. At this Step 102 checkpoint, the evidence supported a reproducible, finite, audited compiler extension from Ω ownership through topology, legal port scheduling, and conditional heterogeneous communication time. Subsequent Steps 103–104 add logical directed-link contention and audited candidate selection (Section 27). Physical-network accuracy, time-optimal scheduling, and compiler reproduction of all four hand-derived attention and Transformer implementations remain open.

27 Directed-link contention and predictive Ω schedule selection (Steps 103–104)

Scope and invariants. The November 3, 2026 extension retains the previously derived MoA ψ-indexed computation, Ω query ownership, owner-rooted logical communication topology, and exact transfer multiset. A transfer is identified by its owner coordinate, parent and child machine coordinates, and payload size. Legal schedules are stage partitions of these transfers respecting receive-before-forward, parent outgoing-port limits, unique delivery, and complete reachability. Stage rearrangement changes neither denotational semantics nor the logical total link bytes. Independent auditors check the schedule against the original topology rather than trusting its constructor.

Conditional contention model (Step 103). For a directed logical link ℓ used in stage s, define $B_{s,\ell} = \sum_{e \in E_{s,\ell}} b_e$, where $E_{s,\ell}$ is the multiset of transfers assigned to that link and stage and b_e is the transfer's payload in bytes. With assumed nonnegative latency α_ℓ and positive bandwidth β_ℓ, the model predicts

$$T_s = \max_{\ell:\, E_{s,\ell} \neq \varnothing} \left(\alpha_\ell + \frac{B_{s,\ell}}{\beta_\ell} \right), \qquad T_{\text{comm}} = \sum_s T_s. \tag{57}$$

Empty stages contribute zero. The sum aggregates simultaneous logical transfers using the same directed link; the maximum assumes concurrency across distinct links within a stage. This is a *conditional logical-link model*, not a calibrated physical DGX network model. It omits unmodeled shared physical paths, protocol overhead, computation overlap, and network congestion beyond the defined directed-link aggregation. The Step 103 implementation and independent contention auditor are preserved in local commit `faf06be`.

Deterministic candidate selection (Step 104). Let $\mathcal{S}_{\text{legal}}$ be the set of schedules admitted by the independent port-schedule auditor. The implementation evaluates a finite candidate subset $\mathcal{C} \subseteq \mathcal{S}_{\text{legal}}$ containing the original deterministic baseline S_0 and twelve alternatives: six priority rules (large payload first, small payload first, slowest individual transfer first, fastest first, link order, and owner order), each with and without a same-directed-link-per-stage exclusion rule. Every candidate is independently audited for schedule legality and contention-time accounting. The selection is

$$S^* \in \arg\min_{S \in \mathcal{C}} T_{\text{comm}}(S), \qquad T_{\text{comm}}(S^*) \leq T_{\text{comm}}(S_0). \tag{58}$$

Ties retain the earlier selected candidate; the baseline is initially selected. The inequality is guaranteed for successfully audited candidates under the specified prediction model, *not* for physical runtime. The candidate subset is heuristic and finite; the procedure does not establish optimality over all legal schedules. The Step 104 implementation and regression tests are preserved in local commit `018bc83`.

Reproducible heterogeneous-link example. For attention descriptor `Work(7,5,4,3)`, machine shape $\langle 2, 3 \rangle$, query ownership counts $\langle 1, 2, 1, 1, 1, 1 \rangle$, and two outgoing ports per parent, all logical links use $\alpha = 2\,\mu$s and $\beta = 10\,$GB/s except the directed link $\langle 1, 1 \rangle \to \langle 1, 2 \rangle$, whose bandwidth is 1 GB/s. The scheduler examines thirteen candidates, selects `slow_first:shared_links_allowed`, and retains six stages, $\langle 12, 8, 4, 3, 2, 1 \rangle$ transfers per stage, and 840 logical link bytes. The independent schedule and contention auditors both pass.

The schedule advances owner $\langle 1, 1 \rangle$'s 24-byte transfer $\langle 1, 1 \rangle \to \langle 1, 2 \rangle$ from stage 1 to stage 0 and postpones its 24-byte transfer to $\langle 1, 0 \rangle$ from stage 0 to stage 1. It also advances owner $\langle 0, 1 \rangle$'s 48-byte transfer $\langle 0, 1 \rangle \to \langle 1, 1 \rangle$ from stage 2 to stage 1, while postponing owner $\langle 0, 0 \rangle$'s 24-byte transfer on the same directed link from stage 1 to stage 2. Consequently, stage 0 becomes $0.0192\,\mu$s slower, while stages 1 and 2 save 0.0240 and $0.0024\,\mu$s. The net predicted saving is $0.0072\,\mu$s (approximately 0.0597% of

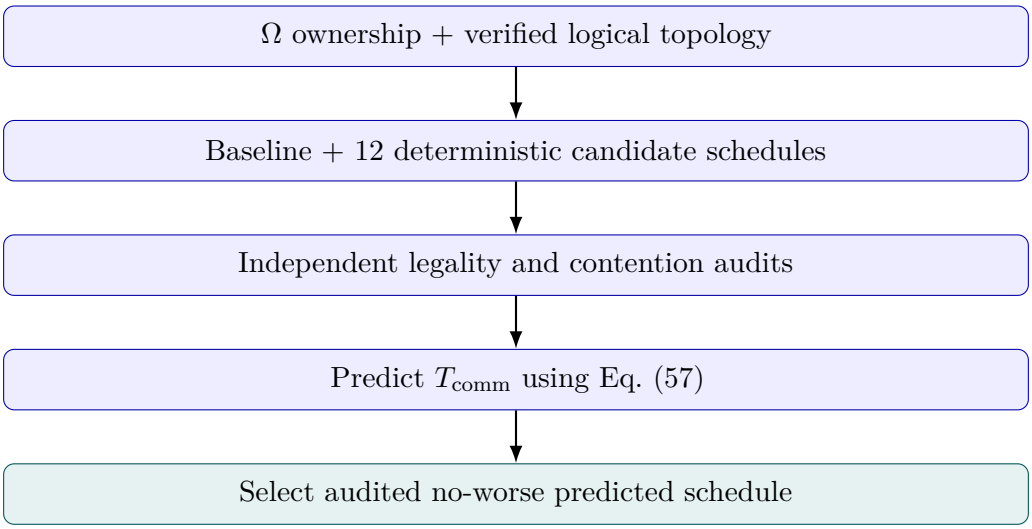


Figure 24: Step 104: MoA-derived machine ownership and logical dependencies constrain the candidate schedules. Independent audits precede model-based selection. This is not hardware-autotuning or an exhaustive optimality proof.

Table 6: Step 104 stage times under the conditional directed-link model. A negative saving denotes a slower selected stage.

Stage	Baseline (μs)	Selected (μs)	Saved (μs)
0	2.0048	2.0240	−0.0192
1	2.0480	2.0240	+0.0240
2	2.0048	2.0024	+0.0024
3	2.0024	2.0024	0
4	2.0024	2.0024	0
5	2.0024	2.0024	0
Total	12.0648	12.0576	+0.0072

baseline). This illustrates why minimizing each stage independently need not minimize the sum under interstage transfer dependencies.

Regression evidence and limits. The Step 103 contention model added 16 tests and the full `tests/universal_ir` suite passed 218 tests. Step 104 added 16 further tests covering the numerical improvement, stage reassignment, determinism, alternative machine shapes, port counts, uniform and heterogeneous links, zero and large latency, and corrupted-result rejection. The November 3 DGX-head-node transcript reports `Ran 234 tests in 2.308s -- OK`. These are software regression results, not measurements of communication time on DGX devices. The four hand-derived attention/Transformer reference implementations remain acceptance targets for automated compiler reproduction. Future work includes bounded exhaustive search with independently checkable optimality certificates for small machine shapes, scaling studies, calibrated physical-network prediction, and actual OpenMP/OpenACC/MPI execution validation.

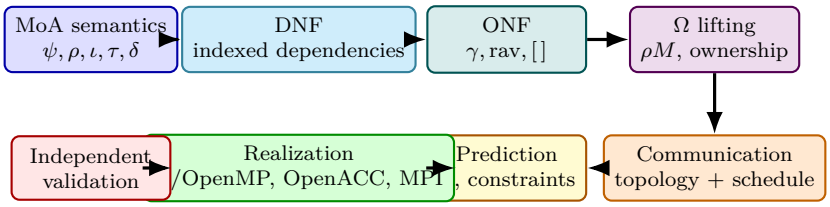


Figure 25: The MoA research program: semantics to DNF, ONF, Ω machine ownership, communication, prospective prediction, realization, and validation. Colors distinguish mathematical, machine-mapping, predictive, and validation responsibilities. The diagram expresses the architecture and the currently implemented bounded components, not a claim that every pass is integrated.

28 From MoA Semantics to Verified Machine-Array Communication

The Mathematics of Arrays (MoA) provides a single mathematical language for specifying shaped-array computations, deriving their indexed behavior, lowering that behavior to addresses, and assigning work and communication to a shaped machine. Our implementation demonstrates portions of this progression through independently checkable compiler components. The current implementation is a *restricted compiler prototype*, not yet a general-purpose MoA compiler. In particular, the earlier attention implementations and their hardware results were derived and programmed by hand; they must not be presented as outputs of this compiler.

28.1 The MoA compilation contract: portable, scalable, verifiable

The central claim of this research is methodological: *one MoA mathematical derivation can be specialized to different shaped machines without changing the meaning of the computation.* Portability follows from separating ψ semantics from γ addressing and Ω ownership; scalability is addressed by expressing the machine itself as an array whose shape and resources may change; verifiability comes from checking invariants at each transformation boundary. None of these properties, by itself, establishes performance portability on all hardware. Instead, performance is a prospective, explicitly parameterized prediction to be validated by measurement.

For an array A, its shape and component count satisfy

$$\rho A = \langle s_0, \ldots, s_{r-1} \rangle, \qquad \delta(A) = \tau(\rho A) = r, \qquad \tau(A) = \prod_{k=0}^{r-1} s_k. \tag{59}$$

Here $\delta(A)$ is a scalar dimensionality, and $\tau(A)$ is the number of components. A rank-one array remains a vector; a rank-two array is a matrix. Index selection obeys the MoA reconstruction identity

$$(\iota(\rho A)\ \psi\ A) \equiv A. \tag{60}$$

The mathematical obligation of Ω lifting is to preserve the logical array and its components while changing the ownership and realization of those components:

$$\tau(A_{\text{logical}}) = \tau(A_{\text{lifted,logical}}), \qquad \rho M = \langle p_0, \ldots, p_{r-1} \rangle. \tag{61}$$

Physical padding or replicated storage is accounted for separately; it must not be silently equated with logical $\tau(A)$.

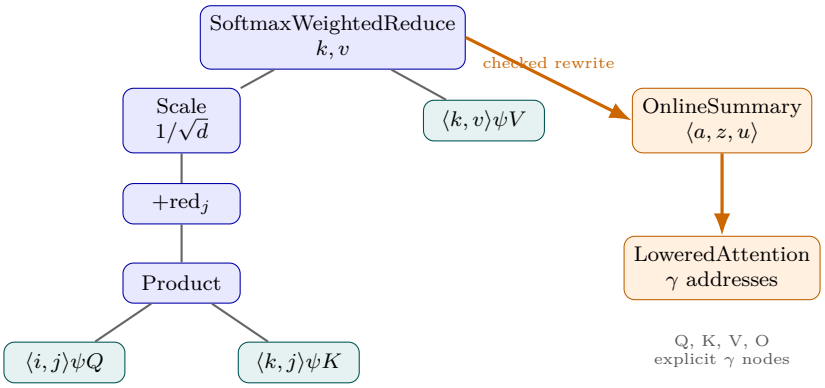


Figure 26: Typed MoA attention abstract syntax tree (AST) and the restricted verified-pattern transformation. The AST preserves ψ dependencies; the right-hand nodes represent the online summary and addressed lowering. Structural checks do not by themselves prove universal floating-point equivalence.

28.2 Typed ψ specification and restricted lowering

The typed forward-attention intermediate representation represents arrays by shape, selections by ψ, and the score expression by a product, reduction, and scale. With $\rho Q = \langle n, d\rangle$, $\rho K = \langle m, d\rangle$, and $\rho V = \langle m, d_v\rangle$, its score dependence is

$$s_{ik} = \frac{1}{\sqrt{d}} \underset{j=0}{\overset{d-1}{+\mathrm{red}}} \left((\langle i, j\rangle \, \psi \, Q) \times (\langle k, j\rangle \, \psi \, K) \right). \tag{62}$$

The corresponding stable softmax-weighted reduction selects $\langle k, v\rangle \, \psi \, V$ over the key index k to produce $\langle i, v\rangle \, \psi \, O$. The implementation recognizes this *specific* typed dependence pattern and converts it to an online summary with state $\langle a, z, u\rangle$. It rejects unsupported expressions rather than claiming a general ψ rewrite system. The summary is subsequently lowered to explicit γ addresses for Q, K, V, and O. For the sequential layout used by this backend,

$$\gamma(\langle i, v\rangle, \rho O) = i d_v + v, \qquad \rho O = \langle n, d_v\rangle. \tag{63}$$

The typed implementation emits C with optional OpenMP and an OpenACC source backend. Source generation alone is not evidence of successful execution on a particular accelerator, nor does its structural verifier prove universal floating-point equivalence.

A typed AST, not a copied kernel. The implemented restricted attention IR contains explicit constructors `Array`, `Psi`, `Product`, `Reduce`, `Scale`, and `SoftmaxWeightedReduce`. The stable online rewrite checks the precise indexed pattern before constructing `OnlineSummary`; `gamma_lower` produces `GammaAddress` nodes. Figure 26 makes these transformations visible.

For each fixed query index i, the stable online summary begins with $a_0 = -\infty$, $z_0 = 0$, and $\langle v\rangle \psi u_0 = 0$. For each key index k, let

$$a_{k+1} = \max(a_k, s_{ik}), \tag{64}$$

$$w_k = \begin{cases} 0, & z_k = 0, \\ \exp(a_k - a_{k+1}), & z_k \neq 0, \end{cases} \qquad f_k = \exp(s_{ik} - a_{k+1}), \tag{65}$$

$$z_{k+1} = w_k z_k + f_k, \tag{66}$$

$$\langle v\rangle \psi u_{k+1} = w_k(\langle v\rangle \psi u_k) + f_k(\langle k, v\rangle \psi V), \tag{67}$$

$$\langle i, v\rangle \psi O = (\langle v\rangle \psi u_m)/z_m. \tag{68}$$

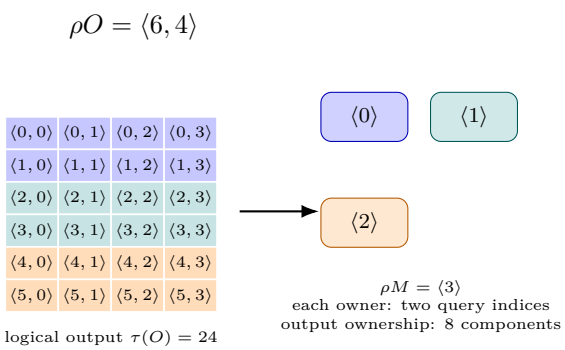


Figure 27: Illustrative Ω lifting of an output of shape $\langle 6, 4 \rangle$ onto a machine of shape $\langle 3 \rangle$. The colors encode disjoint ownership of logical output components, not separate mathematical arrays or physical memory addresses. The implemented bridge checks the corresponding rectangular ownership and γ addressing.

These recurrences explain the code-emission rule and avoid materializing the full score array. They describe real-arithmetic semantics; floating-point order and rounding require their own validation.

28.3 Ω ownership on a shaped machine

Let the target machine be a shaped array M with $\rho M = \langle p_0, \ldots, p_{r-1} \rangle$ and $P = \tau(M) = \prod(\rho M)$. The compiler enumerates machine coordinates using $\iota(\rho M)$ in a specified sequential γ order. For ordinal $q \in \{0, \ldots, P-1\}$, it assigns the half-open query interval

$$I_q = \left[\left\lfloor \frac{nq}{P} \right\rfloor, \left\lfloor \frac{n(q+1)}{P} \right\rfloor \right). \tag{69}$$

Empty intervals are permitted when $P > n$. The verifier checks that these intervals cover every query index exactly once, in order, and preserve the number of output components:

$$\sum_{q=0}^{P-1} |I_q| d_v = n d_v = \tau(O). \tag{70}$$

A multidimensional bridge maps each query interval to the output rectangle $I_q \times [0, d_v)$ at its machine coordinate. It checks output ownership, logical coverage, the machine shape, and sequential γ addresses. Its resource checks concern output storage; replicated inputs, temporary arrays, communication buffers, and physical execution time are outside this bridge's scope. This ownership rule balances query counts but is not a heterogeneous placement optimizer.

28.4 Logical resource and communication certificates

The resource certificate counts, for each machine component, owned output components, score dot products, score multiplications and additions, value accumulations, and online-summary components. These are exact *logical* counts for the specified attention shape and ownership, not measured time, power, energy, or memory traffic.

A separate communication module assumes complete read-only copies of Q, K, and V on each component and an `MPI_Allgatherv` contract that makes the complete output O available everywhere. If component q owns $c_q = |I_q| d_v$ output components, then

$$\sum_q c_q = N, \qquad N = \tau(O). \tag{71}$$

Each component's complete output has N components, while its nonlocal requirement is $N-c_q$. Therefore the aggregate logical nonlocal requirement is exactly

$$\sum_{q=0}^{P-1}(N-c_q)=(P-1)N, \qquad B_{\text{logical}}=(P-1)Nb, \tag{72}$$

where b is the number of bytes per output component. The implementation derives collective counts and displacements from the ownership intervals and cross-checks them against an independent MPI emitter. Equation (72) describes a logical data requirement, *not* the physical bytes, message count, routing, or collective algorithm used by a particular MPI library. It excludes the initial replication cost of Q, K, and V.

28.5 Derived abstract topology and legal stage scheduling

For each nonempty owner interval, the topology module constructs a deterministic breadth-first spanning tree over nearest neighbors of the shaped machine array. Its edges are directed from an already reached component to a newly reached component. A tree for owner q has $P-1$ transfers of its payload of $c_q b$ bytes, giving the abstract link-volume identity

$$B_{\text{tree}}=\sum_{q:c_q>0}(P-1)c_q b=(P-1)Nb. \tag{73}$$

This is a *candidate* allgather realization on an abstract nearest-neighbor machine array, not a reconstruction of the physical DGX network or MPI's chosen collective algorithm.

The port scheduler turns these directed transfers into discrete stages. For stage s, let E_s denote its transfers, and let p_{out} be the outgoing-port limit. Legality requires

$$\forall x\in\iota(\rho M):\quad |\{e\in E_s:\text{parent}(e)=x\}|\le p_{\text{out}}. \tag{74}$$

A component receiving an owner's payload in stage s cannot forward that payload until a subsequent stage. The scheduler is deterministic and greedy; it preserves the required transfer volume and checks eventual delivery, but does not certify minimum stage count. A separate audit checks full topology, dependency, and reachability conditions. The basic scheduler does not model incoming-port restrictions or contention.

28.6 Prediction is distinct from legality and measurement

The later contention-aware modules attach explicit heterogeneous directed-link parameters to legal schedules. Under the stated synchronous-stage model, the predicted communication time is

$$\widehat{T}(S)=\sum_{s\in S}\max_{e\in E_s}\left(\alpha_e+\frac{B_{e,s}}{\beta_e}\right), \tag{75}$$

where α_e is the modeled startup time, β_e is the modeled link bandwidth, and $B_{e,s}$ is the bytes assigned to link e in stage s. This is a conditional prediction, not a measured hardware time.

The independently checked finite-instance result of Step 107 shows that minimizing legal stage count need not minimize (75). In the specified three-component heterogeneous example, a two-stage candidate predicts $40.048\,\mu$s, whereas the exact predicted-time optimizer selects three stages with $20.2008\,\mu$s. The approximately 49.56% reduction is a comparison *within the model*, not an observed DGX speedup. Exact optimization here means exhaustive optimization over the finite legal schedule space for that instance; it is not a claim of universal compiler optimality.

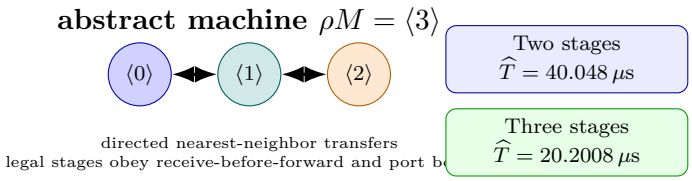


Figure 28: Step 107's finite-instance counterexample on a three-component abstract machine. Under the specified heterogeneous directed-link contention model, the exact minimum-predicted-time schedule uses more stages. These numbers are *predictions*, not hardware measurements.

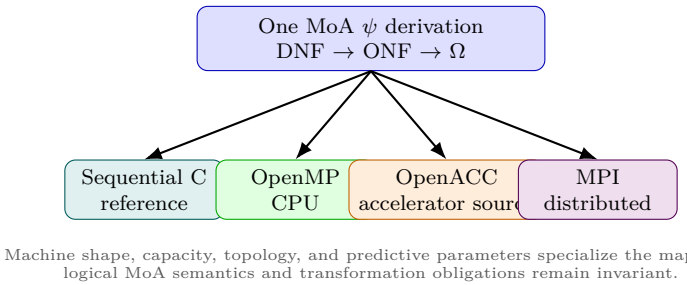


Figure 29: The portable and scalable MoA research goal: a common mathematical source with target-specific realization and independently checkable mapping. Backend coverage and hardware validation are separate empirical questions.

Implementation boundary. Together, these modules provide an auditable path from a restricted typed ψ specification to γ-addressed code, Ω ownership, logical communication, an abstract topology, legal schedules, and conditional predicted costs. The interfaces and certificates establish concrete progress toward a unified MoA compiler. They do not yet constitute a general automatic pipeline from arbitrary source programs to globally optimal physical execution.

28.7 One derivation, multiple realizations

The portability objective is not to disguise hardware differences. It is to express them explicitly as shaped machine properties and mapping constraints, while keeping the MoA ψ specification fixed. A sequential C realization, an OpenMP CPU realization, an OpenACC accelerator-target source realization, and an MPI-distributed realization are alternative backends or realization paths; they are not alternative mathematical definitions of attention. The current codebase contains bounded examples of these paths, but the presence of a backend does not prove that all target machines execute it successfully or with the predicted performance.

Verifiability as a compiler design principle. Each stage must provide a falsifiable obligation: shape legality for ψ; preservation of indexed dependencies in DNF; correct γ addresses in ONF; exact logical ownership under Ω; resource and communication accounting; topology reachability; schedule dependency and port legality; and predictive-model assumptions stated before comparison with measurements. This discipline makes compiler outputs auditable, while keeping open the larger research problem of general semantic equivalence and performance-optimal compilation.

Research horizon. The next milestone is to connect these separately tested components into one automatic compiler driver and require it to reproduce the hand-derived MoA attention and Transformer implementations and their validated behavior. That is a proposed acceptance criterion, not a result already demonstrated by the present restricted prototype.

29 Related work and comparative positioning

Array and compiler foundations. MoA builds on its original array-algebra foundations and subsequent formal and implementation work [3, 4, 5, 6, 7, 8, 9, 10]. It starts from the semantics of shaped arrays, ψ selection, ρ shape, ι index generation, and an explicit passage from DNF to γ-addressed ONF and Ω-lifted machine mappings. This should be evaluated alongside, rather than conflated with, established compiler approaches. The polyhedral model represents affine iteration domains and dependence-preserving schedule transformations [22, 23]; it offers powerful legality and locality tools but is not the same representation as the MoA normal-form derivation. MLIR supplies extensible intermediate representations and lowering infrastructure [20]; it is a plausible implementation substrate, not evidence that a MoA proof or predictor already exists. Halide separates image-processing algorithms from schedules [16], and its autoscheduling work addresses the difficult choice of efficient schedules [17]. TVM combines tensor-expression compilation, hardware-aware optimization, and automated search [18, 19]; Triton supplies a GPU programming and compiler system [21]. None should be characterized as lacking portability or sophisticated scheduling. The distinguishing MoA hypothesis is that one explicit mathematical array semantics can carry checkable index, addressing, mapping, and resource obligations through the entire derivation; its implementation and comparative benefit must be demonstrated, not presumed.

Deep-learning systems and attention kernels. PyTorch [24], TensorFlow [25], JAX [26], and XLA [27] provide important reference points for semantics, differentiation, graph compilation, and execution. Transformer attention originates with Vaswani et al. [28]; memory-efficient implementations include FlashAttention and FlashAttention-2 [29, 30]. Their numerical behavior, memory traffic, fusion, and throughput provide meaningful baselines for an eventual compiler-generated attention implementation. The present paper does not claim to outperform those systems: a fair comparison must align shapes, precision, hardware, compilation, warmup, and end-to-end versus kernel-only measurement. Prior hand-derived MoA attention work [12] establishes a reference derivation, but those implementations must not be counted as compiler-generated outputs.

Portable parallel and distributed realization. OpenMP [31] specifies shared-memory parallel execution and accelerator constructs; OpenACC [32] specifies directive-based accelerator programming; MPI [33] specifies message passing across distributed processes, with Open MPI one implementation [34]. Their roles are complementary. The compiler's desired output is a faithful realization of a common ψ-indexed computation and its γ-addressed, Ω-lifted mapping, not a claim that these programming standards themselves derive MoA semantics. Earlier device-specific source generation remains historical evidence of a restricted backend. Portable scalability must be tested with actual OpenMP/OpenACC/MPI programs and documented toolchains, not inferred from portability of the standards alone.

Communication and performance prediction. BSP [35], LogP [36], and LogGP [37] are established abstractions for communication and synchronization costs; Roofline [38] and communication-avoiding analysis [39] provide complementary resource bounds. The staged link model in Section 24 is a conditional, schedule-specific estimate, not a new general network-performance theorem or a measured replacement for those models. The earlier Step 102 model omits contention; Step 103 adds logical directed-link contention (Section 27), while physical routing and calibration remain unmodeled. The broader predictive program is developed in [14]; agreement with measured time and energy on additional hardware remains a separate validation requirement.

Signal-processing comparisons and provenance. The accepted CSCI 2026 manuscript [15] develops MoA-derived image-processing and attention examples. The original signal-processing experiment [1]

supplies a controlled Halide realization and frozen prospective validation, while the A40 study [2] supplies an MPI/OpenMP-target two-device validation. Section 29.1 reports the actual evidence with its measurement boundaries. Those implementations were hand-derived, not produced by the compiler described in this paper. Thus they motivate a stringent compiler acceptance test: reproduce the mathematical results, executable mappings, and experimentally observed behavior without retroactively claiming earlier work as generated output.

29.1 Experimental evidence from MoA-derived signal and image processing

The companion signal-processing study [1] provides a concrete comparison with Halide as an *execution realization*, rather than treating Halide as an unoptimized foil. The experimental sequence separates pointwise Threshold (no semantic cross-owner communication), 3×3 Sobel (an index-derived two-row halo), and a heterogeneous Halide Sobel realization. The latter used a frozen CPU–GPU ownership split, $q_C = 9537$ and $q_G = 252605$ output rows, with an 8192-byte two-row boundary exchange. These quantities follow from the stated index dependencies and ownership, while measured time also depends on the actual CPU affinity and accelerator execution path.

Table 7: Previously hand-derived signal-processing validation from [1]. Predictions were fixed before the reported validating executions. Times are reported critical-path times for the individual realizations; they are *not* speedup measurements of the present compiler or interchangeable kernel-only benchmarks.

Realization	Frozen (ms)	Measured (ms)	Error (%)
Threshold, heterogeneous	86.205582	85.816671	−0.451
Sobel, OpenMP target	10.050047	10.666868	+6.137
Sobel, Halide	6.495540	6.465319	−0.4653

The Halide experiment reported a checksum of 10,528,147,004, equal to the expected value. Its controlled CPU placement used eight distinct physical cores after an earlier nominal eight-logical-CPU allocation was found to contain simultaneous-multithreading sibling pairs. The measured GPU-rank median of 6.465319 ms formed the critical path; the CPU-rank median was 5.418318 ms. The original study explicitly cautions that resident-device Halide timings and end-to-end OpenMP-target timings have different data-movement boundaries. We therefore compare *prospective prediction accuracy within each realization*, not relative performance across incomparable timing boundaries.

The complementary A40 study [2] derived ownership and communication from the same MoA index semantics, then realized a two-device configuration with MPI ranks and OpenMP target offloading. For its two-GPU pointwise test, the frozen semantic halo prediction was zero bytes; for its two-GPU Sobel test it was 32,768 bytes. The reported executions matched those communication predictions for the tested partition and mapping. These are hand-derived experimental validations, not outputs of the SC27 compiler. They provide independently specified acceptance cases for future automated ψ reduction, γ addressing, Ω lifting, ownership, communication, and source generation.

Why the Halide comparison matters. Halide already separates algorithm from schedule [16, 11]. The research distinction is not that Halide cannot optimize image-processing pipelines. Instead, the signal-processing work tests whether MoA index composition, ownership, and measured machine properties can specify required communication and a frozen critical-path prediction before execution. A future compiler-to-Halide or compiler-to-OpenMP/OpenACC/MPI comparison must use matching problem shapes, data residency, hardware, affinity, timing boundaries, and correctness checks. Neither the table above nor the A40 tests establish that the current compiler outperforms Halide.

Table 8: Positioning by primary mechanism; a comparison of research approaches, not a performance ranking.

Approach	Established emphasis	Distinct MoA compiler obligation
Halide / TVM	Algorithm/schedule separation; hardware-aware optimization and search	Derive indexed semantics and explicit addressing/machine-shape certificates
MLIR / polyhedral	IR infrastructure; dependence-aware legal transformation	Express and check ψ reduction, DNF, ONF, and Ω mappings
PyTorch / attention kernels	Framework semantics; optimized attention execution	Reproduce reference attention semantics and measure like-for-like resource costs
OpenMP / OpenACC / MPI	Portable execution and communication mechanisms	Generate faithful realizations and validate correctness and scaling
BSP / LogP / Roofline	Communication and performance abstractions	Predict a specified shaped-machine mapping and validate its error

30 Conclusion

The restricted rewrite pipeline emits CPU, GPU, and coordinated CPU–GPU source from a common attention operation, with subsequent numerical and timing validation on UAlbany DGX hardware (Section 22). The objective is a universal MoA compiler that mechanically derives semantically equivalent coordinated execution across heterogeneous resource ensembles and predicts feasible mappings on a time–energy frontier. MoA provides a principled path from array semantics to operational realization. The scientific contribution pursued here is to automate a derivation chain previously carried out by hand, while maintaining clear correctness and prediction obligations. Initial UAlbany results demonstrate held-out selection and timing validation for a limited empirical model. The logical routing, contention prediction, and candidate-scheduling results establish testable communication structure and conditional time formulas, while OpenMP, OpenACC, and MPI provide the intended portable shared-memory, accelerator, and distributed realization pathways. Historical generated accelerator results remain labeled by their actual implementation; end-to-end OpenACC and MPI performance claims require separately archived tests. Extending deterministic machine-shape theory to predict physical placement, traffic, time, energy, and power remains an open and measurable research objective.

Acknowledgments

We acknowledge University at Albany research-computing support and computational resources used in this research program, including resources accessed through NSF ACCESS. Earlier work used ACCESS resources including Stampede, Expanse, Bridges, Purdue Anvil, and NCSA Delta, including AMD GPU resources on Delta. Separate Chameleon Cloud award support enabled Intel GPU energy experiments. Award numbers, system versions, and experiment-level provenance should be verified against the corresponding records before submission.

References

[1] L. M. R. Mullin et al., “Mathematics of Arrays for Huge Heterogeneous Signal and Image Processing: From Semantic Normal Form to Predictive Machine Mapping,” research manuscript submitted to IPDPS 2027, 2026. The source manuscript provided for this revision documents the heterogeneous Halide Sobel validation; publication status and full author list to be confirmed.

[2] L. M. R. Mullin et al., "MoA A40" two-GPU pointwise and Sobel prospective-validation report, 2026, author-supplied research manuscript. Final title, author list, and publication status to be confirmed.

[3] L. M. R. Mullin, *A Mathematics of Arrays*, Ph.D. dissertation, Syracuse University, 1988. `https://surface.syr.edu/eecs_etd/74/`.

[4] G. Hains and L. M. R. Mullin, "Parallel Functional Programming with Arrays," *The Computer Journal*, vol. 36, no. 3, pp. 238–245, 1993.

[5] L. M. R. Mullin and M. A. Jenkins, "Effective Data Parallel Computation Using the Psi Calculus," *Concurrency: Practice and Experience*, vol. 8, no. 7, pp. 499–515, 1996.

[6] L. Mullin, "A Uniform Way of Reasoning about Array-Based Computation in Radar: Algebraically Connecting the Hardware/Software Boundary," *Digital Signal Processing*, vol. 15, no. 5, pp. 466–520, 2005.

[7] I. A. Grout and L. Mullin, "Realizing Mathematics of Arrays Operations as Custom Architecture Hardware-Software Co-Design Solutions," *Information*, vol. 13, no. 11, art. 528, 2022.

[8] A. Markus and L. Mullin, "Implementing Mathematics of Arrays in Modern Fortran: Efficiency and Efficacy," *Software*, vol. 3, no. 4, pp. 534–548, 2024.

[9] G. Hains and L. Mullin, "New Mathematics for Computer Performance: Array Algebra and Cost Functions," *Mathematics*, vol. 14, no. 9, art. 1479, 2026.

[10] C. Ostrouchov and L. Mullin, "PythonMoA," Quansight Labs, 2022. `https://github.com/Quansight-Labs/python-moa`.

[11] J. Ragan-Kelley et al., "Decoupling Algorithms from Schedules for Easy Optimization of Image Processing Pipelines," *ACM Transactions on Graphics*, vol. 31, no. 4, art. 32, 2012.

[12] L. M. Mullin and G. Hains, "Attention at the Theoretical Minimum: A Mathematics of Arrays Framework for Memory-Optimal Transformer Kernels," arXiv:2606.07713, 2026.

[13] L. M. Mullin and G. Hains, "One Derivation, Any Machine: Deriving a Weighted, Communication-Free Partition for Attention Across a Heterogeneous, Multi-Institution Device Ensemble," 2026.

[14] L. M. Mullin and G. Hains, "Closing the Prediction Gap: Completing Machine Shape So That Predicted Time, Power, Energy, and Mapping Match What Real Hardware Does," arXiv:2610.03197, 2026.

[15] L. M. Mullin, "Mathematics of Arrays: From Invariant Normal Form to Dimension-Lifted Parallel Execution," accepted for CSCI 2026, scheduled for December 2026; publication details pending.

[16] J. Ragan-Kelley et al., "Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines," ACM SIGPLAN PLDI, 2013.

[17] R. Adams et al., "Learning to Optimize Halide with Tree Search and Random Programs," ACM Transactions on Graphics, 2019.

[18] T. Chen et al., "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," USENIX OSDI, 2018.

[19] L. Zheng et al., "Ansor: Generating High-Performance Tensor Programs for Deep Learning," USENIX OSDI, 2020.

[20] C. Lattner et al., "MLIR: Scaling Compiler Infrastructure for Domain Specific Computation," IEEE/ACM CGO, 2021.

[21] P. Tillet, H.-T. Kung, and D. Cox, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," ACM MAPL, 2019.

[22] P. Feautrier, "Some Efficient Solutions to the Affine Scheduling Problem. Part I. One-Dimensional Time," International Journal of Parallel Programming, 1992.

[23] U. Bondhugula et al., "A Practical Automatic Polyhedral Parallelizer and Locality Optimizer," ACM SIGPLAN PLDI, 2008.

[24] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," NeurIPS, 2019.

[25] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," USENIX OSDI, 2016.

[26] J. Bradbury et al., "JAX: Composable Transformations of Python+NumPy Programs," 2018, software project, `https://github.com/jax-ml/jax`.

[27] OpenXLA Project, "XLA: Optimizing Compiler for Machine Learning," project documentation, `https://openxla.org/xla` (accessed 2026).

[28] A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017.

[29] T. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," NeurIPS, 2022.

[30] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," ICLR, 2024.

[31] OpenMP Architecture Review Board, "OpenMP Application Programming Interface," specification, `https://www.openmp.org/specifications/`.

[32] OpenACC Organization, "OpenACC Application Programming Interface," specification, `https://www.openacc.org/specification`.

[33] Message Passing Interface Forum, "MPI: A Message-Passing Interface Standard, Version 4.0," 2021, `https://www.mpi-forum.org/docs/`.

[34] Open MPI Project, "Open MPI: Open Source High Performance Computing," project documentation, `https://www.open-mpi.org/`.

[35] L. G. Valiant, "A Bridging Model for Parallel Computation," Communications of the ACM, 1990.

[36] D. Culler et al., "LogP: Towards a Realistic Model of Parallel Computation," ACM SIGPLAN PPOPP, 1993.

[37] A. Alexandrov et al., "LogGP: Incorporating Long Messages into the LogP Model," Journal of Parallel and Distributed Computing, 1997.

[38] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," Communications of the ACM, 2009.

[39] G. Ballard, J. Demmel, O. Holtz, and O. Schwartz, "Minimizing Communication in Numerical Linear Algebra," SIAM Journal on Matrix Analysis and Applications, 2011.